Waterfall IR Design

Research notes + IR reference

BMA Standard Formulas

May 06, 2026

Waterfall IR Design — research notes + IR reference

Status: Initial design synthesis from 13 prospectus deep reads across RMBS and auto ABS, plus full IR element reference and a worked example. Not a final spec for IR change yet. Part 1 documents what real prospectus language looks like; Part 2 documents the existing IR schema and how to write a deal in it both reusably and readably.

Sample size (13 deals + FNR 2006-018 fixture):

Asset class	Deal	Key features
Agency MBS REMIC	FNR 2006-018 (fixture)	2 collateral groups, PAC + Z + Support, face-weighted support split
Agency MBS REMIC	FNR 2016-104	9 collateral groups, mix of pass-through, sequential, accretion-directed, PAC, face-weighted splits
Agency MBS REMIC	FNR 2019-17	7 collateral groups, **nested face-weighted splits**, **named Aggregate Group** abstraction
Agency Multifamily REMIC	FNMA 2024-M2	Multifamily; structurally similar to single-family REMICs
Agency Synthetic CRT	CAS 2024-R05, CAS 2024-R06	Connecticut Avenue Securities — synthetic credit risk transfer; reference pool of FNMA-acquired loans, M-1 / M-2 / B-1 / B-2 notes, **reverse-seniority bond writedowns** on reference-pool losses (sometimes with later writeup), pro-rata principal pre-stepdown / sequential post-stepdown
Agency MBS REMIC	Ginnie Mae 2025-203	**Confirms the FNR PAC + Z + Support pattern is industry-standard**; "Aggregate Scheduled Principal Balance" same abstraction as Fannie's "Aggregate Group Planned Balance"
Agency MBS REMIC	Ginnie Mae 2025-009 (HECM)	Reverse-mortgage REMIC; **Deferred Interest Amount** (catch-up rule type not in current IR)
Agency MBS REMIC	Ginnie Mae 2024-115 (Multifamily)	Multifamily-specific: **trustee fee % of Principal Distribution Amount** before cascade
Freddie Mac REMIC general OC	(offering circular)	**Single-Tier vs Double-Tier Series**: REMIC-inside-REMIC trust structure (mostly transparent for cashflow IR)
Non-Agency RMBS (subprime)	JPMMT 2006	Single pool, **interest waterfall + principal waterfall** sub-streams, **stepdown date**, **trigger event override**, OC + excess interest, M-1..M-10 mezz with reverse-seniority loss allocation
Non-Agency RMBS (Non-QM)	Verus 2024-9 / 2026-4	**Step-up coupon** at year 5 (time-conditional rate), **LCF (Last Cashflow) class**, mixed pay (senior pro rata, mezz/sub sequential), Class XS for excess spread
Prime Auto ABS	Ford Credit Auto Owner Trust 2024-C	Single pool, **interleaved I/P**, named priority principal amounts, target OC build, reserve replenishment, capped trustee fee with overflow
Prime Auto ABS	Toyota Auto Receivables 2024-A	Same shape as Ford Credit. **Yield Supplement Overcollateralization Amount** for sub-WAC loan adjustment. Confirms prime auto = same grammar across sponsors.
Auto Lease ABS	Toyota Lexus Owner Trust 2024-A (TLOT)	Lease-specific simpler waterfall (8 steps): pro-rata interest across all classes, single combined priority principal amount, "Securitization Value" valuation. Otherwise same grammar.
Subprime Auto ABS	Santander Drive 2024-2 (SDART)	**Same shape as Ford / Toyota prime auto.** Parametric differences (4 mezz classes vs 2-3, 5 named allocations vs 3, $300K trustee cap vs $375K).
Subprime Auto ABS	Westlake 2024-1 (WLAKE)	8-class structure (A-1, A-2, A-3 + B, C, D, E). Same waterfall shape as SDART. Confirms subprime auto pattern is consistent across sponsors.

Key cross-asset finding: every asset class reduces to a small set of structural primitives. Prime auto + subprime auto + auto lease all share one grammar. Agency MBS REMICs across Fannie / Freddie / Ginnie share the same PAC + Z + Support pattern. The existing IR captures most of these patterns; the major real gap is **named computed distribution amounts** (heavy in non-agency RMBS and auto), and the **authoring practice** of fragmenting waterfall steps into one rule per bond (a fixture / generator issue, not an IR limitation).

Asset classes still to cover (not yet in sample): CLOs (managed), marketplace consumer (SoFi / LendingClub / Affirm), equipment ABS, aircraft ABS, solar ABS. Credit card master trusts now covered in Round 3 research (May 2026, see "Credit card master trusts" section in Part 1).

Methodology

For each deal: locate the "Distributions" / "Priority of Payments" section verbatim, categorize each numbered step in the prospectus, and note what abstractions the prospectus implicitly assumes (named amounts, computed amounts, named groups).

RMBS — Agency MBS REMIC

FNR 2006-018 (existing fixture, anchor)

Two collateral groups, Group 1 has PAC + Z + Support classes with a 95.65 / 4.35 face-weighted support split. Group 2 is a sequential cascade with notional IO.

FNR 2016-104 (Dec 2016)

9 separate collateral groups in one trust. Each group has its own mini-waterfall. Sample group structures:

- **Group 1 (Pass-Through):** "Group 1 Principal Distribution Amount to BA until retired."
  - Single rule, single target.
- **Group 4 (Z + Sequential):** Two rules:
  1. "The Z Accrual Amount to A and B, **in that order**, until retired, and **thereafter** to Z."
  2. "The Group 4 Cash Flow Distribution Amount to A, B and Z, **in that order**, until retired."
  - Each rule has *multiple* targets in a sequential list.
- **Group 5 (PAC + Support):** Three numbered phases:
  1. To Aggregate Group **to its Planned Balance**.
  2. To LZ until retired.
  3. To Aggregate Group **to zero**.
  - Phase 1 = PAC schedule cap (`cap_mode=PLANNED`).
  - Phase 2 = pure sequential, no cap.
  - Phase 3 = cleanup (`cap_mode=NONE`).
  - **The whole "PAC + Support" pattern is three numbered rules, not three rules per bond.**
- **Group 6 (Face-Weighted Split):**
  - "The Group 6 Cash Flow Distribution Amount as follows:
    - 67.8420172533% to QA, QV and QZ, in that order, until retired
    - 32.1579827467% to QB, QW and QY, in that order, until retired"
  - Single SPLIT_CASH at the named percentage with two sub-cascades.

FNR 2019-17 (Mar 2019)

7 collateral groups. Adds two patterns we did not see in 2006-018 or 2016-104:

- **Pro-rata pay** as a top-level option: "Group 1 Principal Distribution Amount to A and FA, **pro rata**, until retired." — payment_style=PRO_RATA on a multi-target rule.
- **Nested face-weighted splits** in Group 7:
  - "Group 7 … Aggregate Group III as follows:
    - 16.6666666667% to MA until retired, **and**
    - 83.3333333333% as follows:
      - first, 44.8738331794% to ME until retired, **and** 55.1261668206% to MG and MB, in that order, until retired,
      - second, to MC until retired."
  - The 83% branch is itself a SPLIT_CASH with two sub-streams, AND that split has two sequential phases ("first" / "second").
  - **Recursive structure**: rules can contain sub-rules.

- **Named "Aggregate Group" abstraction**: "Aggregate Group III consists of the MA, ME, MC, MG and MB Classes … has a principal balance equal to the aggregate principal balance of the Classes included in Aggregate Group III."
  - A bond *bundle* with its own planned balance and internal allocation rules. **Not a separate REMIC class** — a virtual grouping for schedule cap and downstream rules.

Common patterns across all 3 agency REMIC deals

- One waterfall per collateral group.
- Each waterfall has 1-3 logical steps.
- **Each step is a multi-target rule**, not one rule per bond.
- Order language is verbatim from the prospectus: "in that order, until retired", "concurrently / pro rata", "to its Planned Balance", "to zero".
- Z-bond accrual is its own pre-step: "Accrual Amount to X until retired, then to Z" (a sequential cascade itself).
- Face-weighted splits are first-class and **can nest**.
- "Aggregate Group" is a named bond bundle with its own schedule.

Agency Synthetic CRT (CAS 2024-R05, CAS 2024-R06)

Connecticut Avenue Securities (CAS) — Fannie Mae's flagship credit risk transfer program. Structurally distinct from cash REMICs but **fully in scope** for the IR.

Structure

- **Reference pool**, not a real cash pool. The trust holds no mortgage collateral — it holds Fannie Mae's payment obligations derived from the *performance* of a reference pool of FNMA-acquired loans. The reference pool's amortization, prepayments, and credit events drive the bonds.
- **Note classes** (typical CAS structure): M-1, M-2, B-1, B-2 plus unrated B-3H. M-1 is most senior of the issued notes; B-2 is most junior. There is also a hypothetical reference tranche stack (A-H, M-1H, M-2H, B-1H, B-2H, B-3H) used to compute payments but with no actual notes.
- **No reserves, no excess spread, no OC.** Credit enhancement is pure subordination — junior notes absorb losses before senior.

Cashflow mechanics

Two simple things happen each period:

1. **Interest** — each note accrues coupon (typically SOFR + spread) on its outstanding balance and Fannie Mae pays that coupon monthly. Like a normal floater.
2. **Principal & writedowns** — the reference pool's scheduled amortization + prepayments are allocated to bonds (pro-rata pre-stepdown, sequential post-stepdown), and any reference-pool credit losses are *written down* against bond balances in reverse seniority (B-2 first, then B-1, M-2, M-1).

What CRT needs from the IR

CRT is the **simplest waterfall structurally** but the **hardest without `BondDef.loss_treatment`**. Because there is no cash collateral, the bonds' economic existence IS their balance — and losses are the primary state change. Specifically:

- **Bond writedown semantics** — when LOSS is allocated to a bond, its balance must decrease and future coupon must accrue on the reduced balance. This is `BondDef.loss_treatment = WRITEDOWN` (proposed addition E in this document).
- **Bond writeup semantics** (rare but real) — if Fannie Mae later determines a credit event was overstated, the writedown is reversed. The bond's balance increases and the bondholder receives a "writeup payment" representing missed coupon. This is `BondDef.writeup_enabled = true`.
- **Stepdown date + performance triggers** — same idea as non-agency RMBS: pre-stepdown is pro-rata, post-stepdown is sequential, but a delinquency trigger reverts to sequential early if collateral underperforms. Expressible with the proposed `WaterfallBranch` (proposed addition C).
- **Reference-pool collateral input** — the IR's existing `Loan` / `DealRunInput` types accept the reference pool's cashflows directly (treated as if they were real); this part works today.

Match with current IR

CRT feature	Current IR	Status
Sequential / pro-rata principal cascade	`PAY_PRINCIPAL` with `payment_style`	✅
Reverse-seniority loss cascade	`PAY_WRITEDOWN` with reverse `to_targets`	✅
Bond balance writedown on loss	none	❌ (need `loss_treatment`)
Bond writeup on loss reversal	none	❌ (need `writeup_enabled` + `PAY_WRITEUP`)

CRT feature	Current IR	Status
Stepdown × trigger conditional waterfall	per-rule `condition_trigger`	⚠️ works but verbose; `WaterfallBranch` is cleaner
Floater coupon	`BondDef.coupon_type=FLOATING` + index	✅

The summary: **CRT is structurally the simplest deal in this research corpus, but the existing IR cannot model it correctly because of the missing bond-level loss treatment.** Fixing that one gap unlocks the entire CRT product family (CAS, STACR, CIRT, ACIS).

RMBS — Non-Agency (Subprime, JPMMT 2006)

Sample read: JPMorgan Mortgage Acceptance 2006 (CWHEQ-style)

This deal is structurally **very different** from agency:

- Two collateral groups (Group 1 / Group 2 mortgage loans).
- **Two parallel waterfalls per period** — Interest Remittance Amount cascade and Principal Remittance Amount cascade — that intersect via OC mechanics.
- 12-step **interest waterfall** plus 5-condition **principal waterfall** (varies by stepdown date and trigger event).
- M-1 through M-10 subordinate stack.
- Excess interest cascade ("Net Monthly Excess Cashflow") with its own multi-step waterfall.
- Net WAC reserve fund with its own sub-waterfall.

Interest waterfall (12 numbered steps)

1. Senior interest (concurrent / pro-rata across A-1A, A-1B, A-2..A-5)
2. Senior unpaid interest shortfalls (concurrent) 3-12. M-1 through M-10 interest, **one step per class**, in strict seniority order.

The prospectus phrases steps 3-12 as one step per mezz class — **NOT** as one rule with [M-1..M-10]. The reason: each step is its own waterfall priority where remaining funds at that step depend on what was paid above. (For interest where shortfalls are tracked per-class, step-by-step is more readable.) **Either rendering (per-class steps OR a single multi-target sequential rule) produces identical math** — but the prospectus authors prefer per-class for clarity in the mezz stack.

Principal waterfall — gated by 2 conditions, 6 conditional blocks

Each block applies under a different **(stepdown date, trigger event)** combination:

- **A:** prior to stepdown OR Trigger Event in effect → Group 1 Principal → Group 1 Certs first, then Group 2 Certs.
- **B:** prior to stepdown OR Trigger Event in effect → Group 2 Principal → Group 2 Certs first, then Group 1 Certs.
- **C:** prior to stepdown OR Trigger Event in effect → remaining combined principal → M-1, M-2, …, M-10 sequentially.
- **D:** post-stepdown AND no Trigger Event → Group 1 Senior Principal Distribution Amount (a different formula) → Group 1 Senior + cross-allocation to Group 2.
- **E:** post-stepdown AND no Trigger Event → Group 2 Senior Principal Distribution Amount → Group 2 Senior + cross-allocation.
- **F:** post-stepdown AND no Trigger Event → remaining → mezz with **Combined Class M-1, M-2 and M-3 Principal Distribution Amount** (a SHARED computed amount that pays all three sequentially).

Computed distribution amounts

Heavy use of NAMED FORMULAS:

- "Group 1 Principal Distribution Amount" = pool collections + advances
- "Group 1 Senior Principal Distribution Amount" = capped formula
- "Combined Class M-1, M-2 and M-3 Principal Distribution Amount" = formula

These are not rules — they are **named scalar calculations** computed each period and referenced by rules. The current IR has `CalculationNode` for trigger metrics but doesn't use it for distribution amounts.

Sequential Trigger Event vs Trigger Event

There are **multiple distinct triggers** controlling different behaviors:

- "Trigger Event" — gates the stepdown.
- "Sequential Trigger Event" — gates Class A-1A / A-1B concurrent-vs-sequential.

Both are computed each period from cumulative loss + delinquency ratios with **time-dependent thresholds** (a different threshold table per Distribution Date range).

Loss allocation — separate cascade

Loss allocation is a SEPARATE waterfall: realized losses go first to M-10 → M-9 → … → M-1 → seniors. **Reverse seniority order**, not the cash distribution order. Currently the IR has no first-class loss-allocation primitive; it's bolted onto bond writedown logic.

Excess interest cascade

"Net Monthly Excess Cashflow" — what's left after paying interest + fees + scheduled principal — has its OWN multi-step waterfall:

1. To OC build until target reached
2. To unpaid interest shortfalls
3. To realized losses (write-up M-10..M-1)
4. To net WAC carryover
5. … and several more

This is a sub-waterfall on a derived stream. The IR's `SPLIT_CASH` primitive can route to a "Net Monthly Excess Cashflow" stream, and then a sequence of rules consumes it. Workable, but requires expressing the cascade explicitly.

Auto ABS — Prime (Ford Credit Auto Owner Trust 2024-C)

Pre-acceleration priority of payments (single 11-step cascade)

```
1. Transaction Fees and Expenses (capped $375K/yr) — to indenture trustee, owner trustee, ARR
2. Servicing Fee — to servicer
3. Class A Note Interest — pro rata across Class A notes
4. First Priority Principal Payment — to Class A, sequentially by class,
   amount = MAX(0, Class A principal − adjusted pool balance)
5. Class B Note Interest
6. Second Priority Principal Payment — to Class A+B, sequentially by class,
   amount = MAX(0, (A+B principal) − adjusted pool balance) − step 4 amount
7. Class C Note Interest
8. Reserve Account Replenishment — to top up to original balance
9. Regular Principal Payment — sequentially by class,
   amount = GREATER OF (a) Class A-1 principal,
                       (b) notes principal − (pool balance − target OC)
   reduced by first + second priority amounts
10. Additional Fees and Expenses (uncapped overflow from step 1)
11. Residual Interest — to depositor
```

Key structural features

1. **Single Available Funds source** — no group split. All collections pool together; the waterfall consumes them in 11 steps.
2. **Interest and principal interleaved** by class seniority, not as separate sub-waterfalls. Step 3 = A interest. Step 5 = B interest. Step 7 = C interest. The 4 / 6 / 9 between are principal acceleration / catch-up steps.
3. **Computed principal amounts**:
   - "First Priority Principal Payment" — explicit formula based on overcollateralization deficit; only nonzero if Class A principal exceeds adjusted pool balance.
   - "Second Priority Principal Payment" — same shape, A+B view.
   - "Regular Principal Payment" — explicit MAX-of-two-formulas with reductions for upstream payments.
4. **Reserve account replenishment as a numbered step** (step 8). The current IR has `PAY_TO_RESERVE` rule type, so this is covered, but the name is hidden from the user in the FNR fixture (no reserve in agency).
5. **Sequential by class for principal**, but **pro rata within class** for interest. Class A has sub-classes A-1, A-2a, A-2b, A-3, A-4; principal pays A-1 first to retired, then A-2a, etc.; interest pays all 5 pro rata each period.
6. **Capped fees with overflow**: Step 1 caps at $375K/year; any excess fees move to step 10 (after principal). The IR's `max_amount_fixed` covers the cap; the carry-to-later-step needs thought.
7. **Post-acceleration priority of payments**: a SEPARATE waterfall for after Event of Default (not detailed in our extract). Adds an ALTERNATE TOP-LEVEL waterfall gated by deal state.

What's notable vs RMBS

- **No PAC / TAC / Z**. Auto deals are pure sequential.
- **Single pool, no groups**.
- **No stepdown gate** the way RMBS has it; instead the auto structure relies on the priority principal mechanism (steps 4 + 6) to maintain seniority backstop, and the "Regular Principal" formula (step 9) to build target OC over time.
- **Reserve account is a first-class participant** in the waterfall, not just a credit enhancement footnote.

• **Asset-rep fees** (a relatively recent addition tied to ABS Rule 15Ga-1) appear as a discrete payee.

Auto ABS — Subprime (Santander Drive Auto Receivables Trust 2024-2)

Pre-acceleration priority of payments (single 14-step cascade)

```
1.  first       — trustee + Delaware trustee + owner trustee + ARR fees
                  ($300K/yr cap)
2.  second      — servicing fee
3.  third       — Class A interest, pro rata
4.  fourth      — First Allocation of Principal
5.  fifth       — Class B interest
6.  sixth       — Second Allocation of Principal
7.  seventh     — Class C interest
8.  eighth      — Third Allocation of Principal
9.  ninth       — Class D interest
10. tenth       — Fourth Allocation of Principal
11. eleventh    — reserve replenishment (to Specified Reserve Balance)
12. twelfth     — Regular Allocation of Principal
13. thirteenth  — trustee fee overflow (per-annum cap exceeded)
14. fourteenth  — residual to certificateholders, pro rata
```

Same shape as prime auto with parametric differences only

The waterfall is structurally **identical** to Ford Credit prime auto. The differences are entirely parametric:

Feature	Ford Credit (prime)	SDART (subprime)
Step count	11	14
Mezz classes	A, B, C	A, B, C, D
Named principal allocations	First, Second, Regular	First, Second, Third, Fourth, Regular
Trustee fee per-annum cap	$375K	$300K
Residual recipient	“Holder of residual interest”	“Certificateholders, pro rata”

Implication for the IR: prime + subprime auto share **one waterfall grammar**. The IR does not need a “subprime auto” rule type or a “prime auto” rule type — same primitives, more parameters.

Post-acceleration priority of payments

After Event of Default + acceleration, a SEPARATE waterfall applies (typically: trustee fees → all classes interest pro rata → all classes principal sequentially with no priority principal acceleration). Triggered by indenture conditions, not period-by-period.

This is structurally the same as the JPMMT “Trigger Event in effect” branch in the RMBS principal waterfall — an alternate mode gated by a deal-state trigger.

Cross-asset-class observations

What ALL deals have in common

1. **Numbered priority of payments** — the prospectus is always a numbered list of steps.
2. **Each step has multiple targets in a list with explicit order semantics** (sequential, pro-rata, concurrent).
3. **Each step has explicit cap semantics**: “to its planned balance” / “until retired” / “to zero” / “amount equal to…”
4. **Triggers gate behavior**, but the *type* of trigger varies: stepdown date + cumulative loss + delinquency in RMBS; acceleration / event of default in auto.
5. **A “named amount” abstraction** is always implied — every deal defines named formulas like “Group X Principal Distribution Amount”, “First Priority Principal Payment”, “Net Monthly Excess Cashflow”, and refers to them by name in subsequent steps.
6. **Residual / equity / depositor** is always the final step.

What VARIES across asset classes

Feature	Agency MBS	Non-Agency RMBS	Prime Auto
Distinct collateral groups	YES (1-9 per deal)	YES (typically 1-2)	NO
Separate INT/PRIN sub-streams	YES (`ACT_INT` / `ACT_PRIN`)	YES (Interest Remittance Amount + Principal Remittance Amount)	NO (combined Available Funds)
Group-aware allocation	YES	YES (Group 1 Certs, Group 2 Certs)	N/A

Feature	Agency MBS	Non-Agency RMBS	Prime Auto
PAC / TAC / Z behavior	YES	NO	NO
Stepdown date gate	NO	YES	NO
OC / Reserve account in waterfall	RARE (REMIC trust-level fees)	YES (excess interest cascade)	YES (step 8)
Sequential vs pro-rata switch	NO	YES (Sequential Trigger Event)	NO
Interleaved I/P by class	NO	NO (separate I and P waterfalls)	YES (interest 3, prin 4; interest 5, prin 6…)
Loss allocation cascade	rare	YES (reverse seniority)	rare (covered by OC)
Computed (named) distribution amounts	LIGHT (just "Group N Cash Flow Distribution Amount")	HEAVY (every step references named formulas)	MEDIUM (priority principal payments are formulas)
Recursive splits	YES (FNR 2019-17 Group 7)	RARE	NO
Aggregate Group bond bundles	YES	NO	NO

Cross-cutting needs

1. **Multi-target rules with explicit payment_style** — already supported in IR. **Use it more** in the FNR fixture and the irGenerator.
2. **Named distribution amounts** — referenced by name in rule sources. Currently the IR has `ACT_INT` / `ACT_PRIN` as built-in streams and `SPLIT_CASH` to create new ones; this generalizes to "any named formula". Need a `CalculationNode`-style "ComputedAmount" object that produces a per-period scalar usable as a `from_source` cap.
3. **`if / elif / else` over rule blocks** — the RMBS principal waterfall has six mutually exclusive blocks (A through F) keyed on (stepdown date, trigger event). Currently expressed via `condition_trigger` on each rule, which works but is verbose and error-prone (every rule must remember to invert the condition for the "else" branch). A `WaterfallBranch` node with ordered `if / elif / else` cases reads exactly like the prospectus.
4. **Bond-level loss treatment** — the existing `PAY_WRITEDOWN` rule already does the reverse-seniority cascade. What's missing is the bond's *response* to a writedown: does the bond's balance decrease (`WRITEDOWN`) or stay constant with a deferred carryover (`NOTIONAL_HOLD`)? Critical for CRT (writedown is the primary economic event), important for non-agency RMBS subs.
5. **Aggregate Group abstraction** — a NAMED collection of bonds treated as a unit (own planned balance, internal allocation rule). Currently expressed by tagging each bond and listing them in a rule's `to_targets`; works but loses the "this is a group" intent.
6. **Reserve account integration** — already supported via `PAY_TO_RESERVE` / `PAY_FROM_RESERVE_*`. Used heavily in auto; present in non-agency RMBS via OC mechanics.
7. **Excess interest sub-cascade** — a separate sub-waterfall driven by what's left after the "main" interest cascade. Need a clean way to express the carry-forward.

Gaps in the current IR

After this 4-deal sample:

Gap	Severity	Affects
Named distribution amounts (computed scalars) referenced by source name	**HIGH**	non-agency RMBS, prime auto
Multi-target sequential rules under-used in fixtures and irGenerator	**HIGH**	every asset class (visual / authoring)
Aggregate Group bond bundles	MEDIUM	agency MBS
Recursive / nested splits	MEDIUM	agency MBS (FNR 2019-17 pattern)
Bond-level loss treatment (writedown vs notional-hold)	**HIGH**	CRT, non-agency RMBS, future CMBS
Conditional waterfall blocks (`if / elif / else` over rule groups)	MEDIUM	non-agency RMBS, CRT, auto post-acceleration
Net WAC reserve / sub-cascade plumbing	LOW (covered by SPLIT_CASH + accounts)	non-agency RMBS
Post-acceleration alternate waterfall	LOW (covered by triggers)	auto
Capped fee with overflow to later step	LOW	auto

Proposed IR additions (DRAFT — not approved)

A. Multi-target rule consolidation (no schema change, fixture rewrite + synth/irGen change)

Stop fragmenting by emitting one rule per bond. Both the FNR fixture authors and the `emitPacTacSchedule` block generator should emit multi-target rules (`to_targets: [PA, PB, PC, PD, EO]`, `payment_style: SEQUENTIAL`, `cap_mode: PLANNED`).

Why first: zero schema risk, biggest immediate benefit. The rule count drops from ~24 to ~5 for FNR Group 1.

B. ComputedAmount node (small schema add)

```
class ComputedAmountNode(BaseModel):
    name: str           # "Group 1 Principal Distribution Amount"
    expression: str     # "principal + advance_prin + recovery"
    description: str
```

Rules can reference `ComputedAmountNode` names in `from_sources` the same way they reference `ACT_INT` / `ACT_PRIN`. The runtime resolves the name to the per-period scalar.

C. WaterfallBranch — `if / elif / else` over rule blocks

The natural way to express "if Trigger Event in effect: pay rules 1; else: pay rules 2" is exactly that — an explicit `if / elif / else` chain at the IR level, not per-rule `condition_trigger` tags. Per-rule tagging works mathematically but is error-prone (you have to remember to invert the condition on every "else" rule and keep the inversion in sync) and unreadable (the prospectus phrasing "If X, do Y, else do Z" is fragmented across many rules).

```
class WaterfallBranch(BaseModel):
    branch_id: str
    description: str
    cases: list[WaterfallCase]   # ordered; first matching case fires

class WaterfallCase(BaseModel):
    when: str | None             # trigger name, None for the `else` arm
    invert: bool = False         # treat the trigger as `not when`
    expr: str | None             # OR a free expression evaluated per period
    rules: list[RuleNode]        # rules to execute when this case matches
```

Reads exactly like the prospectus:

```
waterfall_rules:
  - branch_id: principal_waterfall
    description: "RMBS principal allocation: stepdown × trigger event"
    cases:
      - when: TriggerEvent
        rules: [ ... rules block A: sequential to senior, no mezz ... ]
      - when: StepdownDate
        invert: true
        rules: [ ... block B: pre-stepdown sequential ... ]
      - expr: "stepdown_date_reached and not trigger_event"
        rules: [ ... block C: post-stepdown pro-rata ... ]
      - when: null   # the `else` arm
        rules: [ ... default block ... ]
```

Why prefer this over per-rule `condition_trigger`:

1. **Mutual exclusion is explicit.** `if/elif/else` semantics guarantee exactly one branch fires. Per-rule tags can accidentally fire two branches if conditions aren't perfectly complementary.
2. **DRY.** Don't repeat the same condition on 14 rules.
3. **Reads like the prospectus.** "Block A — Trigger Event in effect" maps to one case; "Block B — pre-stepdown" to another.
4. **Refactor-friendly.** Adding a rule to a conditional block is one append; under the per-rule scheme it's an append plus matching the condition exactly.

When to still use per-rule `condition_trigger`: one-off gates on a *single* rule (e.g., "this single fee only applies if the servicer is in default"). Per-rule remains for one-off cases; `WaterfallBranch` is for multi-rule blocks.

This is the IR equivalent of asking "why don't we just write `if/else`?" — the answer is "we should, and that's what this node is."

D. AggregateGroup bond bundle

```
class AggregateGroupDef(BaseModel):
    name: str           # "Aggregate Group III"
```

```
    members: list[str]  # ["MA", "ME", "MC", "MG", "MB"]
    schedule_contract: list[...] | None    # planned balance (PAC)
    internal_allocation: PaymentStyle      # SEQUENTIAL / PRO_RATA
```

A virtual bundle that rules can target by name. The runtime caps at the bundle's planned balance and distributes internally per the allocation rule.

E. Bond-level loss treatment (the real loss-allocation question)

Reverse-seniority loss allocation isn't a special new IR construct — it's the standard pattern across RMBS, CRT, and future CMBS, and the existing `PAY_WRITEDOWN` rule type already expresses it:

```
rule_type: PAY_WRITEDOWN
from_sources: [LOSS]
to_targets: [B-2, B-1, M-2, M-1]   # reverse seniority
payment_style: SEQUENTIAL
```

The **real question** that the IR currently doesn't answer is: **when a loss hits a bond, what happens to that bond's balance and accrual base going forward?** Two distinct treatments exist in real prospectuses:

1. **`WRITEDOWN`** — bond balance is reduced by the loss amount. Future coupon accrues on the *reduced* balance. Future principal distributions are based on the reduced balance. This is the CRT default and the non-agency RMBS subordinate default.
2. **`NOTIONAL_HOLD`** — bond balance stays unchanged. The loss becomes a deferred-amount carryover that reduces cash to that bond until covered by recovery / excess interest, but the bond continues to accrue coupon on its *full* original balance. This is rare but appears in some prime jumbo deals and in particularly investor-friendly subordinate structures.
3. **`NONE`** — bond doesn't absorb losses at all (typically senior-most class with full guarantee).

Adding this is a small `BondDef` extension:

```
class LossTreatment(StrEnum):
    WRITEDOWN = "WRITEDOWN"
    NOTIONAL_HOLD = "NOTIONAL_HOLD"
    NONE = "NONE"

class BondDef(BaseModel):
    ...
    loss_treatment: LossTreatment = LossTreatment.NONE
    writeup_enabled: bool = False   # CRT-style loss reversal
```

The runtime change is small: in `PAY_WRITEDOWN`, look up each target bond's `loss_treatment` and either decrement its balance (`WRITEDOWN`) or accumulate a deferred amount (`NOTIONAL_HOLD`). Recovery / writeup applies inversely.

Why this matters for CRT. CRT bonds are pure notional-bearing instruments — there is no actual cash collateral in the trust. The bonds' economic existence IS their balance, and loss allocation IS the primary cashflow event. Without `loss_treatment` on `BondDef`, the IR can't model CRT at all. The existing `PAY_WRITEDOWN` rule type is the *cascade*; the missing piece is the *bond's* response to that cascade.

F. Recursive SPLIT_CASH (no schema change, runtime change)

Already supported in principle: a SPLIT_CASH target stream can be the source of another SPLIT_CASH. Verify this works for FNR 2019-17 Group 7's nested 16.67 / 83.33 / first / second pattern.

Round 3 schema review — separating bond identity from rule behavior

This round addresses an architectural concern surfaced during review: the current schema **conflates three different concepts** on `BondDef` and in the rule type enum:

1. **What a bond IS** (intrinsic identity that doesn't change with rules) — IO vs PO vs cash-pay vs Z vs residual vs pseudo
2. **What a bond HAS** (intrinsic schedules / properties) — coupon, notional, schedule contract, maturity, tracking relationships, loss treatment
3. **HOW a bond gets paid** (extrinsic, lives on the rule that pays it) — sequential vs pro-rata, cap mode, conditional gating

The dividing line: if the property is true regardless of which waterfall is paying the bond, it belongs on `BondDef`. If it describes a particular rule's allocation behavior, it belongs on the rule. The current schema crosses this line in several places.

G. Collapse TrancheType + TrancheBehavior into a single TrancheKind

Current state. `TrancheType` has 13 values (`SEQUENTIAL`, `PAC`, `PAC_II`, `TAC`, `SUPPORT`, `Z_BOND`, `ACCRETION_DIRECTED`, `FLOATER`, `INVERSE_FLOATER`, `IO`, `PO`, `PSEUDO`, `RESIDUAL`). `TrancheBehavior` has 4 (`SEQUENTIAL`, `PAC`, `TAC`, `Z`). They overlap, and both encode rule-behavior properties on the bond.

The 13 values are mixing five orthogonal concepts:

Concept	Values currently in `TrancheType`	Belongs on
Cashflow identity	`IO`, `PO`, `RESIDUAL`, `PSEUDO`, `Z_BOND`	`BondDef` (the bond IS this)
Schedule type	`PAC`, `PAC_II`, `TAC`	`BondDef.schedule_contract` (already has `schedule_type`)
Coupon style	`FLOATER`, `INVERSE_FLOATER`	`BondDef.coupon_type` (already exists)
Payment role	`SUPPORT`	derived from `support_tranches` relationships
Allocation order	`SEQUENTIAL`, `ACCRETION_DIRECTED`	rule's `payment_style` + bond's accretion targets

Proposed. A single `TrancheKind` that answers only "what *kind* of bond is this":

```
class TrancheKind(StrEnum):
    CASH_PAY = "CASH_PAY"   # ordinary bond — pays cash interest + cash principal
    PAC      = "PAC"        # planned amortization class — REQUIRES schedule_contract
    TAC      = "TAC"        # targeted amortization class — REQUIRES schedule_contract
    IO       = "IO"         # interest-only, notional-bearing
    PO       = "PO"         # principal-only, zero coupon
    Z        = "Z"          # accrues during accretion phase, then pays cash
    RESIDUAL = "RESIDUAL"   # equity / sweep
    PSEUDO   = "PSEUDO"     # accounting sink (fees, residuals tracking quantities)
```

8 kinds. **Validation rule**: if `kind ∈ {PAC, TAC}` then `schedule_contract` MUST be non-empty; if `kind ∈ {CASH_PAY, IO, PO, Z, RESIDUAL, PSEUDO}` then `schedule_contract` MUST be empty (or absent). The bond's identity drives the schedule requirement, not the other way around — Pydantic enforces this with a model validator.

Why keep PAC and TAC as kinds (not just "CASH_PAY with a schedule"):

- **The prospectus uses these names.** "Class PA is a PAC Class. Class TA is a TAC Class." The IR vocabulary should match.
- **They are distinct economic identities.** A PAC has a *band* of speeds it tolerates; a TAC has a *single* target speed and no upside protection. Both concepts have to be captured somewhere; making them kinds is more legible than inferring from `schedule_speed_low == schedule_speed_high`.
- **Validation enforcement.** A PAC bond without a schedule is always wrong. The kind makes that constraint explicit and catchable at IR validation time, not at runtime.

Everything else moves out:

- **SUPPORT** → derived: a bond is "support" if some other bond's `relations` list points to it as `SUPPORTED_BY`. The bond itself is just a `CASH_PAY`. Removing the SUPPORT enum value doesn't lose information.
- **PAC_II** → still a `PAC` kind. The "II" is a structural ordering relative to PAC_I (PAC_I shortfalls cascade to PAC_II's planned balance), expressible via the rule's order and a relation pointing PAC_II at PAC_I. Doesn't need to be a separate kind.
- **FLOATER / INVERSE_FLOATER** → already lives in `coupon_type` enum (`FIXED | FLOATING | INVERSE_FLOATING | ZERO`). Duplicate.
- **SEQUENTIAL** → not a property of the bond. It's the rule's `payment_style`.
- **ACCRETION_DIRECTED** → encoded by Z's `ACCRETES_TO` relation (see proposal H below).

`TrancheBehavior` is fully redundant with the simplified schema and should be deleted.

H. Unify tranche relationships into one structured list

Current state. Three different fields express bond-to-bond structural relationships:

- `support_tranches: list[str]` — PAC's support stack (the bonds that absorb prepay variability for me)
- `supported_by_tranches: list[str]` — Z's accretion targets (where my PIK accrual is directed while they're outstanding)
- `parent_tranche: str | None` + `relation_type: StructureRelation`
  - `notional_ratio: float | None` + `tracks_bonds: dict[...]` — a tangled mix used for IO/PO notional tracking and inverse-floater parenting

`StructureRelation` has only 3 values (`floater_inverse`, `io_po`, `z_accrual`), which doesn't cover the real-world set of relationships you raised: POs, IOs, **inverse IOs, inverse floaters, super floaters**, MACR exchanges.

Proposed. One `tranche_relations` list with a typed enum that covers every real-world relationship:

```
class TrancheRelationType(StrEnum):
    SUPPORTED_BY        = "SUPPORTED_BY"        # PAC -> support stack
    ACCRETES_TO         = "ACCRETES_TO"         # Z -> bonds receiving Z's PIK
    NOTIONAL_TRACKS     = "NOTIONAL_TRACKS"     # IO / inverse IO (notional follows other bond's balance)
    BALANCE_TRACKS      = "BALANCE_TRACKS"      # PO mirroring another bond's principal
```

```python
    COUPON_INVERSE_OF   = "COUPON_INVERSE_OF"   # inverse floater pegged to a floater
    COUPON_LEVERAGE_OF  = "COUPON_LEVERAGE_OF"  # super floater (multiple of an index)
    MACR_EXCHANGE       = "MACR_EXCHANGE"       # exchangeable / combinable

class TrancheRelation(BaseModel):
    relation_type: TrancheRelationType
    targets: list[str]              # bonds at the other end of the relationship
    weights: list[float] | None     # for tracking aggregate baskets
    leverage: float | None          # for COUPON_LEVERAGE_OF (e.g., 2.5x)
    cap: float | None               # for COUPON_INVERSE_OF (cap − rate ∗ leverage)
    floor: float | None
    description: str = ""

class BondDef(BaseModel):
    ...
    relations: list[TrancheRelation] = []
```

This single field covers:

Bond type	`relations` example
Plain IO	`[{relation_type: NOTIONAL_TRACKS, targets: [PA, PB, PC, PD], weights: [...]}]`
Inverse IO	Same as IO, plus a separate `coupon_type=INVERSE_FLOATING`
Plain PO	`[{relation_type: BALANCE_TRACKS, targets: [...]}]` (or just stand-alone with `coupon_type=ZERO`)
Inverse Floater	`[{relation_type: COUPON_INVERSE_OF, targets: [Floater], cap: 0.10}]`
Super Floater	`[{relation_type: COUPON_LEVERAGE_OF, targets: [Index], leverage: 2.5, cap: 0.12}]`
PAC support stack	PAC has `[{SUPPORTED_BY, targets: [WA, WB, ...]}]`
Z bond	Z has `[{ACCRETES_TO, targets: [TA, TB]}]`
MACR / RCR	`[{MACR_EXCHANGE, targets: [...], weights: [...]}]`

The 4 separate fields collapse into 1, gaining expressiveness for inverse IOs / super floaters / MACR while staying compact.

I. Coupon as a value or a schedule (not just a scalar)

Current state. `BondDef.coupon: float | None` plus separate `cap` / `floor` scalars. Cannot express:

- Step-up coupons (Verus 2024-9 — coupon goes from `c` to `c + 1.00%` at month 60)
- Step-down coupons
- Lockout-period coupons (initial fixed rate, then floats)

Proposed. Allow each rate field to be either a scalar or a period-keyed schedule:

```python
RateOrSchedule = float | list[RateScheduleEntry]

class RateScheduleEntry(BaseModel):
    from_period: int        # inclusive
    rate: float

class BondDef(BaseModel):
    ...
    coupon: RateOrSchedule | None
    margin: RateOrSchedule | None      # floater spread
    cap: RateOrSchedule | None
    floor: RateOrSchedule | None
```

Most bonds keep the simple scalar form. Step-up bonds use:

```yaml
coupon:
  - { from_period: 1,  rate: 5.50 }
  - { from_period: 60, rate: 6.50 }
```

J. Notional, not "size"

Current state. `size_dollars: float | None` and `size_pct: float | None`.

Proposed rename. "Size" is ambiguous. Use `notional`:

```
notional: float | None                    # dollar notional / par balance
notional_pct_of_collateral: float | None  # 0..1 of pool original balance
```

For IOs the notional is the literal IO notional (no principal flow). For cash-pay bonds the notional is the par/face balance. Same field, semantically clearer.

K. Two-phase schedule derivation — speeds are design-time, schedule_contract is runtime

The right framing: **schedule_contract is a derived artifact**. The user owns the schedule's *inputs* (speed band, prepay model, collateral assumptions); the system computes and caches the *output* (the per-period planned balance vector); the runtime reads only the output.

```
┌─ Design-time (structuring) ──────────────────┐   ┌─ Runtime (execution) ─┐
│                                              │   │                       │
│  user inputs:                                │   │  reads:               │
│    schedule_speed_low     (PAC low, e.g., 100%)  │    schedule_contract  │
│    schedule_speed_high    (PAC high, e.g., 250%) │                       │
│    schedule_model_type    (PSA / CPR / ABS)  │   │  uses it via:         │
│    collateral_assumptions (WAC, WAM, balance)│   │    cap_mode=PLANNED on│
│                           │                  │   │    the PAY_PRINCIPAL  │
│                           ▼                  │   │    rule               │
│                derive_schedule(...)          │   │                       │
│                           │                  │   └───────────────────────┘
│                           ▼                  │
│  cached output:                              │
│    schedule_contract: list[{period, balance}]│
│                                              │
└──────────────────────────────────────────────┘
```

Derivation algorithm (PAC):

For each period *t*, compute the projected outstanding balance of the bond at speed_low and speed_high using the prepay model and collateral assumptions. The PAC's planned balance at *t* is the **maximum** of the two (or, equivalently, the balance under the slower speed) — this is what gives PAC stability across the band: fast-prepay scenarios drain support tranches; slow-prepay scenarios let support tranches absorb the shortfall.

For TAC: same calculation, single speed, balance = projected balance at that speed.

Schema after Round 3:

```python
class BondDef(BaseModel):
    ...
    kind: TrancheKind                              # PAC | TAC | CASH_PAY | ...

    # Design-time inputs (kept so the schedule is re-derivable)
    schedule_speed_low:  float | None              # PAC low end; TAC uses this as the target
    schedule_speed_high: float | None              # PAC high end; TAC sets == low
    schedule_model_type: PrepayModelType           # PSA | CPR | ABS | CUSTOM_VECTOR
    schedule_tolerance_bps: float | None

    # Runtime canonical (what the cap_mode=PLANNED rule consults)
    schedule_contract: list[ScheduleEntry]         # [{period, target_balance}]
```

The redundant `schedule_speed_target` field is dropped — `schedule_speed_low == schedule_speed_high` already encodes TAC as the degenerate band.

When does schedule_contract get re-derived?

- User changes the speed band in the structuring UI.
- User changes prepay model or collateral assumptions.
- User changes the bond's notional (changes the absolute balance curve).
- User adjusts which support tranches absorb shortfalls (changes the schedule shape under stress).

The structuring UI runs derivation eagerly on input change; the runtime never derives — it only consumes. This separation gives us:

- **Reproducibility**: a deal definition with a populated `schedule_contract` runs identically every time, regardless of whether speeds / collateral assumptions changed *after* the schedule was built. The IR is self-contained.
- **Auditability**: the schedule_contract is what was *actually used* in pricing and execution. Speeds are inputs; the schedule is the audit artifact.
- **Performance**: derivation only happens at design time; the runtime does not pay derivation cost on every period.

- **UI ergonomics**: the user works in speed-band terms ("100%- 250% PSA"), which is how prospectuses and analysts talk; derivation handles the math.

Validation contract:

- If `kind ∈ {PAC, TAC}` and `schedule_contract` is empty → invalid IR (can't run a PAC bond without a schedule).
- If `kind = PAC`, both `schedule_speed_low` and `schedule_speed_high` should be set.
- If `kind = TAC`, both should be set and equal.
- The validator can additionally check that `schedule_contract` is *consistent* with the speeds and collateral (re-derive and compare with tolerance). This is an optional integrity check, not a correctness requirement at runtime.

L. Drop CONCURRENT from PaymentStyle — already a fact

Verified. `PaymentStyle` is already two values:

```
class PaymentStyle(StrEnum):
    SEQUENTIAL = "SEQUENTIAL"
    PRO_RATA   = "PRO_RATA"
```

`CONCURRENT` was never in the code; the IR-reference table earlier in this doc listed it incorrectly. **No code change required.** The doc references to `CONCURRENT` have been removed.

M. Reserve rules are not a separate rule type; they're rules sourced from / targeted to accounts

Current state. Five reserve-related rule types:

- `PAY_TO_RESERVE` — deposit into reserve
- `PAY_FROM_RESERVE` — generic withdrawal
- `PAY_FROM_RESERVE_INTEREST` — withdrawal labeled "interest"
- `PAY_FROM_RESERVE_PRINCIPAL` — withdrawal labeled "principal"
- (plus the same shape for recourse: `PAY_RECOURSE_INTEREST`, `PAY_RECOURSE_PRINCIPAL`)

This conflates **two orthogonal things**: what cash is moving (interest? principal?) and where it's coming from / going to (account? bond? stream?). Every "from reserve" rule is really "`PAY_INTEREST` (or `PAY_PRINCIPAL`) with `from_sources` set to a reserve account."

Proposed. Three rule types covering all account interactions:

```
PAY_INTEREST       # to: bond. from: any stream OR any account.
PAY_PRINCIPAL      # to: bond. from: any stream OR any account.
PAY_TO_ACCOUNT     # to: account. from: any stream OR any account.
                   # (renamed from PAY_TO_RESERVE — accounts are not just reserves)
```

The current `PAY_FROM_RESERVE_INTEREST` becomes `PAY_INTEREST` with `from_sources=[ReserveAcct]`. The current `PAY_TO_RESERVE` becomes `PAY_TO_ACCOUNT`. Recourse is just a named source stream (`from_sources=[RECOURSE_LINE]`).

This change makes accounts truly first-class. Any rule can deposit to one or withdraw from one.

Risk. The runtime *might* attach extra semantics to the reserve-specific rule types (e.g., maintaining a "shortfall carryover" counter that's affected only by `PAY_FROM_RESERVE_INTEREST`). That semantic is worth preserving but should be moved to the rule's effect, not its type — e.g., a `tracks_carryover_for: str` field on the rule that names which bond's interest-shortfall ledger to update. This needs runtime verification before code change.

N. Drop alias tokens; rename INT_CASH / PRIN_CASH — LANDED (Phase 1c, May 2026), superseded by COLL_*

Original proposal (preserved for history): rename `INT_CASH` → `CASH_INT` and `PRIN_CASH` → `CASH_PRIN` for prefix consistency, drop `COLLATERAL` and `GROUP_<id>_COLLATERAL` aliases.

What actually landed (Phase 1c, May 2026): the user's explicit "full BMA-native across the board" directive shifted the rename target to match `BMAActualCashflow` field names directly:

Token	Original proposal	Landed in Phase 1c
`CASH`	keep	keep (= `BMAActualCashflow.act_cash`)
`COLLATERAL`	drop	dropped
`INT_CASH`	rename to `CASH_INT`	renamed to **`ACT_INT`** (= `BMAActualCashflow.act_int`)
`PRIN_CASH`	rename to `CASH_PRIN`	renamed to **`ACT_PRIN`** (= `BMAActualCashflow.act_prin`)
`LOSS`	keep	keep (= `BMAActualCashflow.prin_loss`)
`GROUP_<id>_CASH`	keep	keep

Token	Original proposal	Landed in Phase 1c
`GROUP_<id>_COLLATERAL`	drop	dropped
`GROUP_<id>_INT_CASH`	rename to `GROUP_<id>_CASH_INT`	renamed to `GROUP_<id>_ACT_INT`
`GROUP_<id>_PRIN_CASH`	rename to `GROUP_<id>_CASH_PRIN`	renamed to `GROUP_<id>_ACT_PRIN`
`GROUP_<id>_LOSS`	keep	keep

Rationale for the BMA-native naming over `CASH_*`: every token now maps to exactly one named field on the cashflow dataclass — no translation layer between IR vocabulary and the engine's data model.

Forward direction — `COLL_*` taxonomy (proposed Phase 2): after the CC research validated the need for finer-grained cashflow decomposition (servicer advances, recoveries, loss decomposition, balance refs), the user agreed to advance to a full `COLL_*` prefix scheme that:

- Re-prefixes the four core streams: `CASH` → `COLL_CASH`, `ACT_INT` → `COLL_INT`, `ACT_PRIN` → `COLL_PRIN`, `LOSS` → `COLL_LOSS`.
- Adds 11 decomposition tokens (`COLL_PRIN_SCH`, `COLL_PRIN_PPAY`, `COLL_PRIN_REC`, `COLL_PRIN_ADV`, `COLL_PRIN_ADV_REIMB`, `COLL_INT_ADV`, `COLL_INT_ADV_REIMB`, `COLL_LOSS_PRIN`, `COLL_LOSS_INT`, `COLL_LOSS_ADV`, `COLL_SVC`).
- Adds 3 read-only balance refs (`COLL_PERF_BAL`, `COLL_FCL_BAL`, `COLL_TOTAL_BAL`).

See "Built-in source/target tokens" in Part 2 for the full vocabulary. Phase 1c stops at the BMA-native rename; the `COLL_*` forward step is sequenced after the Proposal R PAIRED-input migration (Phases 1d–1h) lands.

O. AccountType is a label, not runtime semantics

Current `AccountType` enum: `RESERVE | PREFUNDING | REVOLVING | PAYMENT | SPREAD_ACCOUNT`.

(The earlier IR reference in this doc listed the wrong values — `CUSTODIAL | DISTRIBUTION` — and has been corrected.)

Verified against the runtime: `account_type` is purely a display label. The runtime touches it in exactly two places, both of which are passthrough:

1. **Initialization** (`runtime.py:324`) — copy `account_def.account_type.value` (a string) into `AccountWorkspace.account_type`.
2. **Output** (`runtime.py:1611`) — copy that string into the `DealAccountRow` for output / reporting.

There is **no `if account_type == X` branch anywhere** in the runtime. The runtime treats every account uniformly: a named bucket with `balance`, `deposit`, `withdrawal`, and a `required_minimum` array.

`MinimumBasis` (after the Round 3 Q fix) IS the actual behavioral driver for account floors and starting amounts — the runtime now branches on it correctly. `AccountType` remains a passthrough display label.

This means real-world account variety:

- **Reserve account** — credit / liquidity reserve
- **Prefunding account** — holds cash before bonds buy collateral
- **Revolving account** — master-trust style revolving funding
- **Payment / collection account** — temporary holding
- **Spread account** — excess spread tracking
- **Capitalized interest account** — capitalized interest during prefunding
- **Yield supplement account** — auto YSOC
- **Trustee fee reserve** — fee carve-out

…is supported today not by adding more `AccountType` values, but by configuring the right `minimum_basis`, the right `starting_amount`, and the right rules that deposit to / withdraw from the account. For example:

Real-world account	How it's expressed today
Reserve floored at 0.5% of original collateral	`minimum_basis: COLLATERAL_BALANCE`, `minimum_pct: 0.5`, plus a `PAY_TO_RESERVE` rule
Prefunding account drawing down to zero	`starting_amount: <prefunded $>`, `minimum: 0`, plus a deposit rule funded by the prefunding stream and a withdrawal rule that pays it out as principal
Capitalized interest account auto-amortizing	`starting_amount: <cap-i $>`, `minimum: 0`, plus a `PAY_INTEREST` rule with this account as `from_sources` for the bond's coupon during the cap-i period
YSOC account	A `SPREAD_ACCOUNT` (label) feeding a `SPLIT_CASH` that boosts under-WAC bond cash

Proposed change: rename `AccountType` → `AccountCategory` to reflect that the field is a UI / reporting label and stop signaling "this might gate behavior" via the name. No runtime change needed.

If a future deal needs runtime behavior keyed on account type (e.g., "PREFUNDING accounts auto-amortize without explicit rules"), that should be added as a *separate* explicit field — e.g., `auto_amortizes: bool`, `amortization_schedule: list[...]` — not by overloading `account_type` with hidden semantics.

Q. Implement `minimum_basis` and `starting_basis` per-period semantics — FIXED

Status: Implemented. `runtime.py` now honors all 4 basis values for both `minimum_basis` and `starting_basis`. New regression tests in `tests/test_account_minimum_basis.py` lock in the per-period behavior. The remainder of this section is preserved for context.

Original state (silently buggy). The IR had `MinimumBasis` and `starting_basis` fields with 4 enum values:

```
class MinimumBasis(StrEnum):
    FIXED_DOLLAR        = "FIXED_DOLLAR"
    COLLATERAL_BALANCE  = "COLLATERAL_BALANCE"
    NOTE_BALANCE        = "NOTE_BALANCE"
    ORIGINAL_COLLATERAL = "ORIGINAL_COLLATERAL"
```

Intended semantics (per the field name and the per-period `required_minimum` array shape):

Value	Intended behavior
`FIXED_DOLLAR`	`floor = minimum_amount` (constant $)
`COLLATERAL_BALANCE`	`floor[t] = minimum_pct × pool_balance[t]` (steps down as pool amortizes)
`NOTE_BALANCE`	`floor[t] = minimum_pct × outstanding_note_balance[t]` (steps down as bonds amortize)
`ORIGINAL_COLLATERAL`	`floor = minimum_pct × original_pool_balance` (constant)

Actual runtime behavior (`runtime.py:310-321`, `_allocate_account_workspace`):

```
minimum = float(account_def.minimum_amount or 0.0)
if account_def.minimum_pct is not None:
    minimum = max(minimum, collateral_balance_0 * float(account_def.minimum_pct) / 100.0)
required_minimum = np.zeros(cf_len)
required_minimum[:] = minimum
```

The runtime:

1. Computes the minimum **once** at initialization
2. Always uses `collateral_balance_0` (the *original* balance) as the percentage basis
3. Broadcasts the single value across all periods
4. **Never reads `minimum_basis` to decide how to compute**

So `minimum_basis = COLLATERAL_BALANCE` and `minimum_basis = ORIGINAL_COLLATERAL` produce identical runtime behavior. Same for `NOTE_BALANCE` (silently treated as `ORIGINAL_COLLATERAL`).

The `starting_basis` field has the same problem — `starting_pct` always uses `collateral_balance_0`, ignoring the basis enum.

Practical impact. A reserve account authored with `minimum_basis=COLLATERAL_BALANCE` and `minimum_pct=0.5%` expecting the floor to amortize down with the pool will instead have the floor fixed at `0.5% × original_balance` for the deal's entire life. For long-amortizing pools this is a meaningful overstatement of the reserve floor late in the deal's life and can mask reserve breaches.

Proposed fix. Implement per-period recomputation by honoring the enum value. The fix lives at the start of each period's account-evaluation pass (or hoisted into a derived array if the basis value depends only on bond/pool state):

```
def _period_account_minimum(
    account_def: AccountDef,
    period: int,
    pool_balance_t: float,
    note_balance_t: float,
    collateral_balance_0: float,
) -> float:
    pct = account_def.minimum_pct or 0.0
    floor_pct = {
        MinimumBasis.FIXED_DOLLAR:        0.0,
        MinimumBasis.COLLATERAL_BALANCE:  pool_balance_t * pct / 100.0,
        MinimumBasis.NOTE_BALANCE:        note_balance_t * pct / 100.0,
        MinimumBasis.ORIGINAL_COLLATERAL: collateral_balance_0 * pct / 100.0,
    }[account_def.minimum_basis]
    return max(account_def.minimum_amount or 0.0, floor_pct)
```

Same shape for `starting_basis` at period 0 only.

Why this matters. Real-world reserve mechanics step down with amortization in many deals — auto ABS reserves often have a "step-down floor" that's `0.50% of current pool balance with a $X minimum"; non-agency RMBS reserves track note balance through the OC waterfall. Currently neither is correctly modeled.

Severity. This was a HIGH-severity silent correctness bug because the IR accepted the value, the validator passed, the runtime silently ignored the user's choice, and the output looked plausible but was wrong for any account with a non- `ORIGINAL_COLLATERAL` basis.

Resolution (May 2026). Fixed in `runtime.py`. The new `_allocate_account_workspace` builds `required_minimum` per period based on `minimum_basis`:

- `FIXED_DOLLAR` — broadcast `minimum_amount` to every period.
- `ORIGINAL_COLLATERAL` — broadcast `max(minimum_amount, minimum_pct × collateral_balance_0)`.
- `COLLATERAL_BALANCE` — vectorized `max(minimum_amount, minimum_pct × pool_balance[t])` per period using the projected pool balance series.
- `NOTE_BALANCE` — period 0 from the initial note stack at allocation time; periods 1..T are filled by `_refresh_note_balance_minimums(deal, accounts, bonds, t)` inside the main run loop, which sums `bonds[*].balance[t]` for `is_bond=True, is_pseudo=False` bonds and applies `minimum_pct`.

`starting_basis` is honored at period 0 in the same way.

Test coverage in `tests/test_account_minimum_basis.py`:

- `FIXED_DOLLAR` floor stays constant; `pct` is ignored.
- `ORIGINAL_COLLATERAL` floor stays constant at `pct × initial_pool`.
- `COLLATERAL_BALANCE` floor monotonically decreases with pool amortization.
- `NOTE_BALANCE` floor monotonically decreases with bond amortization and tracks the live note stack.
- `minimum_amount` (dollar floor) takes precedence when it exceeds the percentage-derived floor.
- `starting_basis` produces the right initial balance for every enum value.

P. Schema cleanup migration table

Current	Proposed	Rationale
`TrancheType` (13 values)	`TrancheKind` (8 values: CASH_PAY, PAC, TAC, IO, PO, Z, RESIDUAL, PSEUDO)	Drop SEQUENTIAL, SUPPORT, ACCRETION_DIRECTED, PAC_II, FLOATER, INVERSE_FLOATER (rule behavior, derived role, or coupon-type properties); KEEP PAC and TAC because they're real economic identities and validation can enforce that PAC/TAC require `schedule_contract`
`schedule_speed_target` field	(deleted)	Redundant — TAC uses `schedule_speed_low == schedule_speed_high` as the degenerate band
`TrancheBehavior` (4 values)	(deleted)	Fully redundant with `TrancheKind` + `schedule_contract` + `pay_mode`
`support_tranches`, `supported_by_tranches`, `parent_tranche`, `relation_type`, `notional_ratio`, `tracks_bonds`	`relations: list[TrancheRelation]`	One typed list covers PAC support, Z accretion, IO/PO tracking, inverse floater, super floater, MACR
`coupon: float`, `cap: float`, `floor: float`, `margin: float`	`RateOrSchedule` (scalar OR period-keyed schedule)	Step-up / lockout / time-conditional coupons
`size_dollars`, `size_pct`	`notional`, `notional_pct_of_collateral`	Naming clarity (covers IO notional and cash-pay par)
`schedule_speed_low`, `schedule_speed_high`, `schedule_speed_target`	`schedule_speed_band: tuple[float, float]` (or move to `schedule_metadata`)	TAC = degenerate PAC band
`PaymentStyle.CONCURRENT`	(deleted)	Synonym of `PRO_RATA`
`RuleType.PAY_FROM_RESERVE`, `PAY_FROM_RESERVE_INTEREST`, `PAY_FROM_RESERVE_PRINCIPAL`, `PAY_RECOURSE_*`	(deleted)	Use `PAY_INTEREST` / `PAY_PRINCIPAL` with account or recourse stream as `from_sources`
`RuleType.PAY_TO_RESERVE`	`PAY_TO_ACCOUNT`	Accounts are not just reserves
Built-in token `COLLATERAL`	(deleted)	Alias for `CASH`
Built-in token `GROUP_<id>_COLLATERAL`	(deleted)	Alias for `GROUP_<id>_CASH`

Current	Proposed	Rationale
`INT_CASH` / `PRIN_CASH` (and group variants)	**Phase 1c (May 2026):** `ACT_INT` / `ACT_PRIN`. **Phase 2 (planned):** `COLL_INT` / `COLL_PRIN` under the full `COLL_*` taxonomy	Phase 1c aligns IR with `BMAActualCashflow` field names; Phase 2 adds 11 decomposition tokens + 3 balance refs. See proposal N updates and Part 2 "Built-in source/target tokens"
`AccountType` enum (treated as runtime-significant)	`AccountCategory` (UI label only)	**Verified**: runtime never branches on `account_type`
~~`minimum_basis` and `starting_basis` ignored by runtime~~	Per-period recomputation honoring the enum value	**FIXED (May 2026)** — proposal Q. `_allocate_account_workspace` + `_refresh_note_balance_minimums` in runtime.py; covered by `tests/test_account_minimum_basis.py` (9 tests, all passing).

Net schema impact:

- 2 enums collapsed (`TrancheType` 13→8 as `TrancheKind`, `TrancheBehavior` deleted)
- 6 fields on `BondDef` collapsed into 1 list (`relations`)
- 1 BondDef field deleted (`schedule_speed_target`)
- 5 rule types deleted (4 reserve, 1 recourse — folded into the generic interest/principal rules)
- 1 rule type renamed (`PAY_TO_RESERVE` → `PAY_TO_ACCOUNT`)
- 1 enum value deleted (`PaymentStyle.CONCURRENT`)
- 4 built-in tokens cleaned up (drop 2 aliases; rename 2 + group variants)
- 1 enum repurposed (`AccountType` → `AccountCategory` UI label)

The result is a schema where:

- A bond is one of 8 things (kinds), with PAC/TAC carrying a validator that requires a `schedule_contract`.
- Schedules / coupons / notionals / relationships live on the bond as concrete data, not behavior tags.
- Schedule derivation is design-time; the runtime consumes the derived `schedule_contract` directly without re-deriving.
- Behavior — sequential vs pro-rata, schedule-cap vs cleanup, account-sourced vs cash-sourced — lives entirely on the rule.

Round 3 fresh review (May 2026) — grounded against runtime

Before any code change, every Round 3 proposal was re-verified against the actual runtime + schemas. Findings:

#	Proposal	Verdict	Notes
G	Collapse `TrancheType` (13) + `TrancheBehavior` (4) → `TrancheKind` (8)	Confirmed	Runtime uses `tranche_type == "RESIDUAL"` (1 site) and `tranche_behavior == "Z"` (3 sites). Replaceable with `kind == TrancheKind.RESIDUAL` / `kind == TrancheKind.Z`. PAC/TAC stay as kinds; validator enforces `schedule_contract` non-empty.
H	Unify 6 BondDef relationship fields into one `relations: list[TrancheRelation]`	Confirmed with refinement	`support_tranches` (PAC support), `supported_by_tranches` (Z accretion), `tracks_bonds` (IO/PO mirror — used in `ops.py`), `parent_tranche` + `relation_type: StructureRelation` + `notional_ratio` (existing `StructureRelation` enum is `FLOATER_INVERSE | IO_PO | Z_ACCRUAL` — only 3 values). Migration to a 7-value `TrancheRelationType` covers IO/inverse-IO/super-floater/MACR with no behavior loss. Big migration in tranche_behaviors.py and the IO/PO ops.
I	Coupon as `RateOrSchedule` for step-ups	Confirmed	New feature; runtime currently treats coupon as scalar. Touches coupon evaluation + Verus-style step-up bonds.

#	Proposal	Verdict	Notes
J	Rename `size_dollars` / `size_pct` → `notional` / `notional_pct_of_collateral`	Confirmed	Read in 4 places at workspace allocation (`runtime.py:190-254`). Pure rename + fixture migration.
K	Two-phase derivation; drop `schedule_speed_target`	**Confirmed AND already implemented**	`schedule_derivation.py` already exists at `src/bma_cfengine_app/orchestrator/deals/` with `derive_pac_schedule(...)` (lower envelope of two PSA projections) and `derive_tac_schedule(...)` . Runtime never reads speeds — only `schedule_contract` . Proposal collapses to just dropping the redundant `schedule_speed_target` field.
L	**Drop `PaymentStyle.CONCURRENT`**	**DOC WAS WRONG**	The current enum has only `SEQUENTIAL | PRO_RATA` . `CONCURRENT` was never in the code. **No code change needed.**
M	Reserve / recourse rule consolidation	Confirmed with refinement	`PAY_FROM_RESERVE_INTEREST` decrements bond's `int_shortfall` ledger and accumulates `opt_interest` (`runtime.py:1635-1642`). `PAY_FROM_RESERVE_PRINCIPAL` increments bond principal/balance. `PAY_RECOURSE_*` draws against pseudo-bond capacity. **Full collapse needs a `coverage_mode: NORMAL | INTEREST_SHORTFALL | PRINCIPAL_ACCELERATION` field** to preserve the shortfall-ledger semantics. Recourse pseudo-bonds are also valid `from_sources` since they have a balance to decrement. Recommend phased migration.
N	Drop `COLLATERAL` aliases; rename `INT_CASH` / `PRIN_CASH`	**SHIPPED Phase 1c (May 2026)**	Renamed to `ACT_INT` / `ACT_PRIN` (BMA-native) per user directive, not `CASH_INT` / `CASH_PRIN` . `COLLATERAL` aliases dropped. Phase 2 will advance to full `COLL_*` taxonomy with decomposition tokens (post-CC research May 2026).
O	Rename `AccountType` → `AccountCategory`	Confirmed	Already verified to be a passthrough label only.
Q	Implement `minimum_basis` / `starting_basis` per-period	**SHIPPED** (May 2026)	Done.

PAC/TAC schedule derivation (your earlier question)

> *Are PAC/TAC schedules driven off of collateral amortization or targeted bond amortization? Where is the CPR/PSA speed applied — the bond balance or the collateral balance?*

Answer: the CPR/PSA speed is applied to the **collateral (pool)**, not to the bond. The bond's planned balance schedule is **derived** by running the deal's principal allocation logic against the pool's projected cashflows under each speed in the PAC range.

The current implementation is in `src/bma_cfengine_app/orchestrator/deals/schedule_derivation.py` . The algorithm:

```
1. project_pool_principal(pool_balance, wac, term, psa_low, n)
   → array of pool principal cashflow per period at psa_low
2. project_pool_principal(pool_balance, wac, term, psa_high, n)
   → array of pool principal cashflow per period at psa_high
3. _track_pac_principal(proj_lo, pac_size)
   → PAC absorbs pool principal each period up to its
     remaining balance (sequential PAC-first allocation), under psa_low
4. _track_pac_principal(proj_hi, pac_size) → same under psa_high
5. For each period t:
     target[t] = MIN(pac_principal_lo[t], pac_principal_hi[t])
   → "lower envelope" — the principal amount the PAC is
     guaranteed to receive at any speed within the band
6. schedule_contract = [{period, target_principal} for t]
   (legacy form; newer entries use {period, target_balance}
    where target_balance is the integrated remaining balance)
```

The PAC is guaranteed to amortize at *at least* the schedule rate across the entire PSA band because at each period it receives at least `MIN(pac_lo, pac_hi)` worth of principal. Outside the band the support tranches are exhausted (high speed) or the pool just doesn't deliver enough principal (low speed) and the PAC schedule breaks.

TAC uses a single PSA target rather than a band. Same algorithm, no envelope step. Asymmetric protection — covers extension only, not contraction.

Implication for proposal K: the design-time schedule derivation is ALREADY structured the right way. The bond carries the speeds (PAC band edges or TAC target) as design-time metadata, the structuring tool runs the derivation against collateral projections, and the resulting `schedule_contract` goes into the IR. The runtime just consumes the schedule. The only Round 3 cleanup is dropping the redundant `schedule_speed_target` field (TAC = degenerate band where low == high) and confirming the structuring tool handles re-derivation when bond size, collateral assumptions, or speeds change.

Refinement to proposal M (reserve rule consolidation)

The proposed full collapse needs a `coverage_mode` field to preserve the shortfall-ledger semantic that `PAY_FROM_RESERVE_INTEREST` currently encodes (decrements bond's `int_shortfall`, accumulates `opt_interest`).

```
class CoverageMode(StrEnum):
    NORMAL                  = "NORMAL"                  # default; bond receives current-period interest/principal
    INTEREST_SHORTFALL      = "INTEREST_SHORTFALL"      # covers carryover; decrements bond.int_shortfall
    PRINCIPAL_ACCELERATION = "PRINCIPAL_ACCELERATION"  # extra principal; same balance/principal accumulation as normal pay

class RuleNode(BaseModel):
    ...
    coverage_mode: CoverageMode = CoverageMode.NORMAL
```

Then the migration:

Today	After
`PAY_FROM_RESERVE_INTEREST from=[Reserve] to=[A]`	`PAY_INTEREST from=[Reserve] to=[A] coverage_mode=INTEREST_SHORTFALL`
`PAY_FROM_RESERVE_PRINCIPAL from=[Reserve] to=[A]`	`PAY_PRINCIPAL from=[Reserve] to=[A] coverage_mode=PRINCIPAL_ACCELERATION`
`PAY_FROM_RESERVE from=[Reserve] to=[A]`	`PAY_INTEREST from=[Reserve] to=[A] coverage_mode=NORMAL` (default)
`PAY_RECOURSE_INTEREST from=[RECOURSE_LINE] to=[A]`	`PAY_INTEREST from=[RECOURSE_LINE] to=[A] coverage_mode=INTEREST_SHORTFALL` (recourse pseudo-bonds work as account-like sources)
`PAY_RECOURSE_PRINCIPAL from=[RECOURSE_LINE] to=[A]`	`PAY_PRINCIPAL from=[RECOURSE_LINE] to=[A] coverage_mode=PRINCIPAL_ACCELERATION`
`PAY_TO_RESERVE from=[CASH] to=[Reserve]`	`PAY_TO_ACCOUNT from=[CASH] to=[Reserve]`

Net: -5 rule types, +1 rule type rename, +1 enum, +1 rule field. Phased migration recommended.

R. Direct paired-cashflow input — invert the LDCMA adapter

Current state. The deal runtime accepts an LDCMA-format `CollateralCashflows` Pydantic model (22 dict-keyed arrays: `balance`, `principal`, `interest`, `cashflow`, `loss`, `prepbal`, `defbal`, `recovery`, `principal_sched`, `principal_unsched`, `cpr`, `cdr`, `sev`, `dq`, `surv_fac`, `sched_coupon`, `sched_netcoupon`, `coupon`, `effcoupon`, `sched_balance`, `discount_factor`, `cfdate`).

The BMA engine produces typed cashflow objects:

- `BMAScheduledCashflow` (frozen dataclass) — scheduled cashflow under loan contract terms (no prepays, no defaults)
- `BMAActualCashflow` (frozen dataclass) — performed cashflow with prepays + defaults applied
- `CashFlowPair` — wraps `(scheduled, actual)` for the same loan
- `PortfolioCashflow` in `PAIRED` mode — collection of pairs, the canonical multi-loan / portfolio form

The current input flow is backwards from what the user wants:

```
BMA engine output                     CURRENT FLOW
═════════════════                    ════════════

BMAActualCashflow ─┐
PortfolioCashflow ─┼── adapter ──→ CollateralCashflows ──→ runtime
CashFlowPair      ─┘   (from_*)     (LDCMA dict-keyed)
```

The adapter is doing field-name translation (`perf_bal` → `balance`, `act_int` → `interest`, etc.) on every run.

Proposed flow (R)

```
BMA engine output                     NEW FLOW
═════════════════                    ════════

PortfolioCashflow (PAIRED) ─────→ runtime (consumes natively)
                                    ↑
                                    │
                             (parity tests only)
                                    │
LDCMA collCF dict ──→ ldcma_to_paired ──→ PortfolioCashflow (PAIRED)
```

The deal accepts a `PortfolioCashflow` in `PAIRED` mode directly. LDCMA-format input becomes a parity-test artifact, reached through a small adapter `ldcma_to_paired(...)` that synthesizes a `PortfolioCashflow` from the LDCMA dict.

Decisions captured (May 2026 conversation)

Decision point	Resolution
Canonical input shape	`PortfolioCashflow` in `PAIRED` mode. Multi-group deals: one `PortfolioCashflow` per group (`GroupedPortfolioCashflowInput`).
Use of the scheduled stream	Both — output / audit (scheduled-vs-actual decomposition) AND PAC/TAC schedule derivation.
Internal naming convention	BMA native (`perf_bal`, `act_int`, `act_am`, `vol_prepay`, `prin_loss`, etc.). Runtime aligns with the engine vocabulary; LDCMA-format becomes parity-only.
LDCMA support	Adapter (LDCMA → PAIRED) for parity tests against legacy LDCMA fixtures. Existing forward adapters (`from_actual_cashflow`, `from_portfolio_cashflow`, `from_collateral_dict`) become parity-only.

Schema impact

```python
# New canonical input variants in schemas/input.py:

class PairedCollateralInput(BaseModel):
    """Single-pool deal: one PortfolioCashflow in PAIRED mode."""
    mode: Literal[CollateralInputMode.PAIRED] = CollateralInputMode.PAIRED
    cashflow: PortfolioCashflow  # arbitrary_types_allowed
    model_config = {"arbitrary_types_allowed": True}

class GroupedPairedCollateralInput(BaseModel):
    """Multi-group deal: one PAIRED PortfolioCashflow per group."""
    mode: Literal[CollateralInputMode.PAIRED_GROUPED] = CollateralInputMode.PAIRED_GROUPED
    groups: dict[str, PortfolioCashflow]
    model_config = {"arbitrary_types_allowed": True}

# Old variants stay for parity testing but are deprecated:
```

```python
class PooledCollateralInput(BaseModel):  ...    # LDCMA-format
class GroupedCollateralInput(BaseModel): ...    # LDCMA-format
```

Plus a new `CollateralInputMode.PAIRED` and `PAIRED_GROUPED`.

Runtime impact

`_extract_collateral_arrays(run_input)` is rewritten to dispatch on `mode`:

```python
if mode == PAIRED:
    cf = run_input.collateral.cashflow  # PortfolioCashflow
    actual = cf.aggregate_actual()      # sum BMAActualCashflow across portfolio
    sched  = cf.aggregate_scheduled()   # sum BMAScheduledCashflow across portfolio
    return {
        "perf_bal":   actual.perf_bal,    # was "balance"
        "act_am":     actual.act_am,      # was "principal_sched"
        "vol_prepay": actual.vol_prepay,  # was "principal_unsched"
        "act_int":    actual.act_int,     # was "interest" (gross)
        "svc_billed": actual.svc_billed,
        "prin_loss":  actual.prin_loss,   # was "loss"
        "prin_recov": actual.prin_recov,  # was "recovery"
        "new_def":    actual.new_def,
        "scheduled":  sched,              # available for derivation / output
    }, ...
elif mode == POOLED:                    # LDCMA legacy path
    return _extract_legacy_ldcma_arrays(run_input.collateral.collateral)
```

The runtime's internal `cash_avail`, `interest_avail`, `principal_avail` arrays are populated from the BMA-native fields. Phase 1c (shipped May 2026) updated the user-facing token vocabulary to `ACT_INT` / `ACT_PRIN` matching `BMAActualCashflow.act_int` / `act_prin` exactly.

Adapter impact

```python
# New (parity tests only):
def ldcma_to_paired(
    coll: dict[str, Any],
    *,
    initial_balance: float,
    wac_pct: float,
    term_months: int,
) -> PortfolioCashflow:
    """Synthesize a PortfolioCashflow PAIRED from an LDCMA collCF dict.

    Used to drive parity tests against LDCMA fixtures. The synthesized
    BMAScheduledCashflow uses contract terms; the BMAActualCashflow
    uses the LDCMA arrays. Fields the LDCMA dict doesn't provide
    (svc_billed, fcl, exp_am separately) are stubbed at zero or
    derived from available data.
    """
    ...

# Drop / archive (no longer the primary input adapter):
# - from_actual_cashflow
# - from_portfolio_cashflow
# - from_collateral_dict (becomes ldcma_to_paired's friend)
```

Migration phases (R)

1. Add the new `PairedCollateralInput` + `GroupedPairedCollateralInput` variants alongside the existing LDCMA variants. Tests pass.
2. Add the `_extract_collateral_arrays` PAIRED-mode branch. Pin behavior with a parity test running the same FNR fixture through both PAIRED and LDCMA paths. Tests pass.
3. Migrate the FNR 2006-018 fixture to PAIRED input (it already uses BMA `actual_cashflow_from_loan` as the upstream — switch the adapter call from `from_actual_cashflow` to `BMAActualCashflow → PortfolioCashflow PAIRED` directly). Tests pass.
4. Migrate other fixtures and `deal_library.py` deals.
5. Add `ldcma_to_paired` and write parity tests for the LDCMA fixtures (LDCMA → paired → runtime; produce identical bond cashflows to LDCMA → ldcma → runtime).

6. Internal naming refresh: Phase 1c (May 2026) renamed `INT_CASH` / `PRIN_CASH` → `ACT_INT` / `ACT_PRIN` (BMA-native). Internal arrays `interest_avail` / `principal_avail` were left as-is for now; any future rename would target `act_int_avail` / `act_prin_avail` for consistency. Phase 2 (planned, post-CC research) advances to the full `COLL_*` taxonomy.
7. Deprecate then drop `from_actual_cashflow`, `from_portfolio_cashflow`, `from_collateral_dict` after a clean grace period.

Open questions on R (need user feedback)

The implementation plan needs answers to:

1. **PortfolioCashflow API surface.** `PortfolioCashflow` in PAIRED mode holds `list[CashFlowPair]`. Does the runtime need per-loan resolution, or does it consume the aggregate (sum of all `actual` cashflows + sum of all `scheduled`)? My read: runtime needs the aggregate; per-loan is for engine-layer ops. Confirm or override.
2. **Multi-group routing.** A grouped deal currently has `dict[str, CollateralCashflows]`. New form: `dict[str, PortfolioCashflow]`. Does each group's PortfolioCashflow stay independent (no cross-group ops in the engine), or do we need a higher-level `MultiGroupCashflow` that knows about both?
3. **Internal field naming under N + R combined. RESOLVED Phase 1c (May 2026)**: BMA-native everywhere. Runtime arrays named `perf_bal`, `act_int`, `vol_prepay`, etc. Tokens renamed to `ACT_INT` / `ACT_PRIN` to match `BMAActualCashflow` field names exactly. No translation layer. Phase 2 will advance the user-facing tokens to `COLL_*` (semantic prefix) but keep internal arrays BMA-native.
4. **PAC/TAC re-derivation triggers in the structuring tool.** When does the structuring UI re-run `derive_pac_schedule(...)` to refresh `schedule_contract`?
   - On any change to: speed band edges, PSA model, pool balance, pool WAC, pool term, bond size, support stack composition.
   - Should it be eager (every keystroke updates the schedule) or batched (recompute on Save)?

Open questions for user before any code change

1. **Priority.** Which of (A)-(F) above matter for your near-term deal universe? My read: (A) is must-have *now* (it fixes the visual problem you flagged); (B) and (C) are needed once we start modeling private-label RMBS; (D) is agency-MBS-specific;
   1. is **must-have for CRT** and important for non-agency RMBS subordinates; (F) is already kind of working.
2. **More research breadth.** Do you want me to extend this study to the other asset classes (subprime auto, credit card, CLO, marketplace, equipment, solar) before proposing changes, or are the structural patterns above enough to converge on the abstractions for now and revisit the IR when those asset classes are needed? **My recommendation: do (A) now (no IR risk), park (B)-(F) until we have ≥5 deals per relevant asset class.**
3. **The PAC/TAC/Z question** you raised originally. The IR *intentionally* does not have `PAY_PRINCIPAL_PAC_SCHEDULE` as a distinct rule type — the PAC behavior is encoded as `BondDef.tranche_behavior=PAC` + `BondDef.schedule_contract` + `RuleNode.cap_mode=PLANNED`. This is the right call for the *runtime* (PAC and SEQ have the same allocation logic; only the bond's per-period cap differs). But for the *UI* we should recognize "this rule pays a PAC bond with cap_mode=PLANNED" and render it via the existing `pay_pac_schedule` Blockly block. The synthesizer can detect this from the IR (any rule whose targets include a bond with `tranche_behavior=PAC` and whose `cap_mode=PLANNED` is a PAC schedule rule).
   The IR doesn't need a new rule type for PAC/TAC; the synthesizer + irGenerator just need to recognize the pattern.

Additional deals (round 2 research)

Ginnie Mae REMIC 2025-203 (single-family, multi-group)

Three Security Groups; Group 3 has the canonical PAC + Z + Support structure verbatim:

> ***SECURITY GROUP 3*** *— The Group 3 Principal Distribution Amount and the Accrual Amount will be allocated in the following order of priority: 1. Sequentially, to BP and PB,* ***in that order, until reduced to their Aggregate Scheduled Principal Balance for that Distribution Date*** *2. To Z, until retired 3. Sequentially, to BP and PB,* ***in that order, without regard to their Aggregate Scheduled Principal Balance, until retired***

Identical structure to FNR. "Aggregate Scheduled Principal Balance" is the same abstraction as Fannie's "Aggregate Group Planned Balance". Confirms the FNR pattern is industry-standard for agency REMICs.

Ginnie Mae REMIC 2025-009 (HECM-backed reverse mortgage)

Reverse-mortgage REMIC has a distinct structural feature — the **Deferred Interest Amount**. The 3-step waterfall is:

> 1. *Concurrently, to AI, FA and FB, **pro rata based on their respective Interest Accrual Amounts**, up to the Class Interest Accrual Amount*
> 2. *Concurrently, to FA and FB, **pro rata based on their Class Principal Balances**, in reduction of their Class Principal Balances, up to the Principal Distribution Amount*
> 3. *To AI, until the **Class AI Deferred Interest Amount** is reduced to zero*

Step 3 is unique — it pays accrued-but-unpaid IO interest only *after* the principal cascade. Standard MBS pays IO at the same time as bond interest. **Reverse-mortgage REMICs need a "deferred interest catch-up" rule type or a `cap_mode=DEFERRED` flag.** Not in current IR.

Ginnie Mae REMIC 2024-115 (Multifamily)

> *Allocation of Principal: On each Distribution Date, a percentage of the Principal Distribution Amount will be applied to the **Trustee Fee**, and the remainder of the Principal Distribution Amount (the **"Adjusted Principal Distribution Amount"**) will be allocated, sequentially, to A and B, in that order, until retired.*

Multifamily REMICs **deduct trustee fees from principal** as a percentage *before* the cascade. Single-family agency REMICs don't do this (trustee/servicer fees are netted at the underlying MBS layer). The IR's `PAY_FEE` rule with `basis_type=COLLATERAL_BALANCE` covers this; the multifamily case puts the fee earlier in the cascade.

Freddie Mac REMIC general structure (offering circular)

Freddie Mac REMICs introduce a structural concept that doesn't exist in our current IR: **Single-Tier vs Double-Tier Series**:

- **Single-Tier**: REMIC Certificates represent direct beneficial ownership in *one* REMIC Pool. Cashflows flow pool → classes.
- **Double-Tier**: An **Upper-Tier REMIC Pool** + one or more **Lower-Tier REMIC Pools** stacked. Lower-Tier Classes are themselves **Mortgage Securities** of the Upper-Tier Pool. Cashflows flow pool → Lower-Tier classes → Upper-Tier classes.

This is **REMIC-inside-a-REMIC** structure. Most agency deals abstract this away (the cashflow analyst just looks at the ultimate classes). For our IR, we already model this implicitly — the `Loan` → `BMA cashflow engine` → `DealRunInput` chain treats the underlying agency MBS as a black box delivering a single pool's cashflows to the deal. The lower-tier internal mechanics are out of scope unless we want to model RCR / MACR exchanges.

Also notable: **MACR (Modifiable and Combinable REMIC) Certificates** allow exchange of one class for proportionate interests in another. This is purely an issuance/secondary mechanic (no waterfall implication) and doesn't touch the IR.

Verus Securitization Trust 2024-9 (Modern Non-QM RMBS)

Adds three patterns not in JPMMT 2006:

1. **Step-up coupon at year 5.** "On each payment date beginning in January 2029, the A1, A2 and A3 [classes] will receive the sum of [the deal's] fixed coupon, plus 1.00%."
   - Time-conditional coupon change. Requires the IR to support coupon as a function of period, not a static field.
   - The current `BondDef.coupon: float` does not capture this. `BondDef.coupon_schedule: list[{period, rate}]` would.
2. **Last Cashflow (LCF) class.** "A1A, A1B, A1LCF" — A1LCF gets paid LAST among the seniors. New seniority sub-pattern.
   - LCF works as a normal bond with a junior position within the senior tier; expressible via the existing rule ordering.
3. **Class XS** — explicit excess-spread tracking class. Class XS noteholders also control optional-redemption rights.
   - Excess-spread tracking is just a residual class with extra economic rights; expressible via `tranche_type=RESIDUAL`.
   - Optional-redemption is governance, not waterfall, and is out of scope for the IR.

Pay structure: "senior notes pro rata, mezz + sub sequentially." Same hybrid pattern as JPMMT 2006 post-stepdown, but encoded with no stepdown date — Non-QM deals typically don't have one because the loans amortize quickly.

Toyota Auto Receivables 2024-A (Prime Auto, Toyota)

Confirms the Ford Credit prime auto shape. Differences:

- Has **First Priority Principal Distribution Amount** (Class A catch-up) AND **Second Priority Principal Distribution Amount** (Class A+B catch-up) AND **Regular Principal Distribution Amount** (target OC build) — same three named amounts.
- "**Yield Supplement Overcollateralization Amount**" (YSOC) — a parallel concept used in adjusting the "adjusted pool balance" for sub-WAC loans. Specific to auto deals where some loans have rates below the WAC needed to support the bonds. Pure computation tweak, doesn't change waterfall structure.
- Reserve account replenishment, capped trustee fees with overflow — same as Ford / SDART.
- Confirms prime auto = same grammar across sponsors.

Toyota Lexus Owner Trust 2024-A (Auto LEASE — different shape)

Lease-backed ABS has a SIMPLER waterfall (8 steps, not 11-14):

```
1. Servicing Fee
2. Transaction Fees and Expenses (capped $300K/yr)
3. Note Interest (pro rata across all classes — NOT class-by-class)
4. Note Principal — priority principal distribution amount
   (single combined amount, not per-class)
5. Reserve Account Deposit
6. Note Principal — regular principal distribution amount
7. Additional Transaction Fees and Expenses (overflow)
8. Excess Amounts to certificateholder
```

Differences vs auto loan ABS:

- **Pro-rata interest across ALL classes** (no class-by-class step). Less common in loan ABS.
- **One combined "priority principal distribution amount"** instead of per-class first/second priority. The lease structure pays all notes pro rata for principal, with no class-priority cascade.
- **"Securitization Value"** instead of "Pool Balance" — the lease collateral is valued differently because of residual risk on the vehicles.
- Otherwise structurally the same as auto loan ABS.

The IR already supports this (multi-target PAY_INTEREST with PRO_RATA payment_style; multi-target PAY_PRINCIPAL with PRO_RATA). Just simpler.

Westlake Automobile Receivables Trust 2024-1 (Subprime Auto)

8 classes of notes (A-1, A-2, A-3 + B, C, D, E). Same waterfall shape as SDART — interleaved I/P, named priority allocations, reserve replenishment as a step. Westlake confirms the subprime auto pattern is consistent across sponsors.

Updated cross-asset matrix

After 13 deals across 5 asset classes:

Feature	Agency MBS	Agency CRT	Non-Agency RMBS	Prime Auto	Subprime Auto	Auto Lease
Distinct collateral groups	YES (1-9)	NO (single ref pool)	YES (1-2)	NO	NO	NO
Separate INT/PRIN sub-streams	YES	YES (synthetic)	YES	NO	NO	NO
PAC / TAC / Z behavior	YES	NO	NO	NO	NO	NO
Stepdown date gate	NO	YES	YES	NO	NO	NO
Reserve account in waterfall	RARE	NO	YES	YES	YES	YES
Sequential vs pro-rata switch	NO	YES (gate on perf trigger)	YES	NO	NO	NO
Interleaved I/P by class	NO	NO	NO	YES	YES	NO (pro-rata)
Bond writedown on losses	NO	**YES (core mechanic)**	YES	NO	NO	NO
Bond writeup on loss reversal	NO	YES (rare)	RARE	NO	NO	NO
Named computed distribution amounts	LIGHT	MEDIUM	HEAVY	MEDIUM	MEDIUM	LIGHT
Recursive splits	YES	NO	RARE	NO	NO	NO
Aggregate Group bond bundles	YES	NO	NO	NO	NO	NO
Step-up coupon	RARE	NO	YES (Non-QM)	NO	NO	NO
Deferred interest catch-up	RARE (HECM)	NO	NO	NO	NO	NO
Acceleration alternate waterfall	NO	NO	NO	YES	YES	YES

Updated IR gap list

Adding the new patterns from round 2:

Gap	Severity	Affects	Already in IR?
Multi-target rule consolidation (authoring)	**HIGH**	every class	NO (fixture/irGen issue, not IR)
Named computed distribution amounts	**HIGH**	non-agency RMBS, auto	NO
~~`minimum_basis` and `starting_basis` honored at runtime~~	~~HIGH~~ → **FIXED (May 2026)**	every account-using deal	YES — implemented in `runtime.py`; regression tests in `tests/test_account_minimum_basis.py` (proposal Q)
Aggregate Group bond bundles	MEDIUM	agency MBS	NO
Recursive SPLIT_CASH	MEDIUM	agency MBS	PARTIAL (need runtime verify)
Bond-level loss treatment (`writedown` vs `notional_hold`)	**HIGH**	CRT (core), non-agency RMBS	NO
Conditional waterfall blocks (`if / elif / else`)	MEDIUM	non-agency RMBS, CRT, auto post-accel	PARTIAL (per-rule trigger only)
Bond writeup on loss reversal	LOW	CRT	NO
Time-conditional bond coupons (step-ups)	**MEDIUM**	Non-QM, callable seniors	NO
Acceleration alternate waterfall	**MEDIUM**	every auto deal	PARTIAL (trigger-based)
Deferred interest catch-up (HECM IO)	LOW	reverse mortgage REMIC	NO
Net WAC reserve sub-cascade	LOW	non-agency RMBS	PARTIAL (SPLIT_CASH + accounts)
Capped fee with overflow to later step	LOW	auto	PARTIAL (max_amount_fixed)

Credit card master trusts (round 3 research, May 2026)

Reviewed prospectuses across 5 issuers to validate / extend the collateral-stream taxonomy and IR design before locking in Phase 1c (`COLL_*` token rename) and forward phases:

- **Capital One COMET** (Card series 2002-CC supplement; SF-3 registration statement)
- **Chase Issuance Trust** (CHASEseries A-2024-2, A-2025-1, indenture exhibit 4.3)
- **Citibank Credit Card Issuance Trust** (Citiseries 2023-A2 plus earlier 424B5 filings)
- **Discover Card Execution Note Trust** (DCENT Class A 2022-2)
- **American Express Credit Account Master Trust** (Series 2025-4)

The structures are highly consistent — sponsor differences are mostly cosmetic. What differs from RMBS / auto are the **structural mechanics**, not the cashflow vocabulary.

Stream taxonomy alignment

RMBS / auto	Credit card master trust
`act_int` = pool interest	**Finance Charge Collections** = periodic finance charges + annual membership fees + cash-advance fees + late charges + interchange + recoveries on charged-off receivables + investment earnings + (optional) discount-option amounts
`act_prin` = scheduled + voluntary prepay	**Principal Collections** = scheduled + unscheduled principal repayments allocated at the master-trust level
`prin_recov` = post-FCL liquidation recovery (a *principal* stream)	Recoveries flow into Finance Charge Collections — NOT principal
`prin_loss` = realized principal loss	**Defaulted Amount** / **Investor Charge-Offs** — drives reductions to the per-bond Nominal Liquidation Amount (see gap #2 below)

Token implication: the `COLL_*` taxonomy proposed in proposal N (see Phase 1c) handles all 5 CC issuers without modification. The 4-primitive core (`COLL_CASH`, `COLL_PRIN`, `COLL_INT`, `COLL_LOSS`) plus the 11-token decomposition is sufficient.

The role of *recoveries* flips between RMBS and CC: in RMBS recoveries fold into `COLL_PRIN_REC` (principal stream); in CC they fold into `COLL_INT` (finance-charge stream). This is a loan-level engine concern (CC needs a different cashflow generator than RMBS), not a waterfall-token concern. The token vocabulary is asset-class neutral.

Structural mechanics not yet in IR (8 gaps)

These are *architectural* gaps surfaced by the CC research, separate from naming. Each is orthogonal to the `COLL_*` token discussion; adding them does not require new tokens.

#	Gap	Severity	Examples	Suggested approach
1	**Discount Option** — pre-waterfall reclassification of a % of newly-generated principal as finance charges	MEDIUM	Chase, Capital One, Citi all reference; Chase shows it currently inactive (yield factor = 0)	`deal_knobs.discount_factor` applied by the runtime *before* rule dispatch, OR top-of-waterfall `SPLIT_CASH from=[COLL_PRIN] to=[COLL_PRIN, COLL_INT] weights=[1-d, d]` (requires relaxing the SPLIT_CASH built-in-target rule)
2	**Nominal Liquidation Amount (NLA)** — per-tranche virtual balance, distinct from `BondDef.balance`; reduced by charge-offs and principal-to-interest reallocations, reimbursed by excess spread, caps future allocations	**HIGH** — required for any CC fixture	universal across CC issuers	New `BondDef.nla` field tracked alongside `balance`; runtime decrements/reimburses via dedicated rules
3	**Required / Available Subordinated Amount** — senior-class subordination requirement expressed as % of senior outstanding; “available” = remaining subordinated NLA minus already-used reallocations	HIGH	universal	First-class concept (`required_subordination` per senior bond) OR expression-driven cap. First-class cleaner because every reallocation rule needs to consult it
4	**Multi-series in a master trust** with cross-series sharing (Shared Excess Finance Charge / Shared Excess Principal) — distinct from `collateral_groups`	HIGH	universal	Either each series is its own `DealDefinition` with an inter-deal sharing layer, OR add a `series` concept to `DealDefinition` parallel to `collateral_groups`. Both are major efforts
5	**Funding-account accumulation periods** — Principal Funding Account, Interest Funding Account; bonds receive cash only via these intermediaries; controlled accumulation deposits 1/12 of expected outstanding for the 12 months before bullet payment	MEDIUM	universal (PFA/IFA), AmEx-specific (collateral interest accumulation)	Existing `AccountDef` with `cap_mode=PLANNED` plus a derived “1/N of expected outstanding” schedule. Missing: explicit “accumulation period start” trigger that switches behavior. Could be implemented via existing trigger machinery
6	**Reallocation of Principal Collections to cover senior Interest** — diverts subordinated principal to pay senior interest; reduces subordinate NLA; capped by available subordination	HIGH (ties to #2 + #3)	universal	Falls out from #2 + #3. Express as `PAY_INTEREST from=[COLL_PRIN] to=[A_X]` with a `max_amount_expr` against subordinated NLA
7	**Three-month rolling-average triggers** — excess-spread trigger, portfolio-yield trigger, base-rate trigger, pay-rate trigger	MEDIUM	universal (early-amortization triggers)	Extend `TriggerMetricType` with `*_3MO_AVG`, `*_NMO_AVG` rolling-window variants

#	Gap	Severity	Examples	Suggested approach
8	**Pay-out / Early Amortization deal-state machine** — deal switches from revolving / accumulation mode to early-amortization mode on event triggers; principal then pays directly to bonds in priority	MEDIUM	universal	Either a deal-state enum with transitions OR a single `EARLY_AMORTIZATION` flag computed via `DealCalculation` and gated through every relevant rule's trigger. Verbose but expressible today

Cross-issuer stylistic differences (minor, no IR impact)

Feature	COMET	Chase	Citi	Discover	AmEx
Class structure	A/B/C/D	A/B/C	A/B/C	A/B/C/D	A/B + collateral interest
Interchange separated from finance charges	Folded	Folded	Folded	Sometimes	**Yes**
Discount option	Available	**Active**	Available	Available	Available
Multiple tranches per class	Yes	Yes	Yes	Yes	Single-issuance series
Cross-series sharing groups	Yes	Yes	Yes	Yes	Yes
Class C reserve (excess spread driven)	Yes	Yes	Yes	Yes	Different mechanism (collateral interest)
Subordination via separate notes vs internal interest	Notes	Notes	Notes	Notes	**Collateral interest** (legally a residual position)

AmEx's "collateral interest" is functionally equivalent to a Class C/D note — just structured legally as a depositor-held residual position rather than a securitized note. Renames the role; doesn't change IR mechanics.

Implications for phase plan

CC fixtures cannot be modeled at the *waterfall* level until at least gaps 2, 3, 6 land (NLA + required subordination + reallocation machinery). Gap 4 (multi-series) blocks fully realistic master-trust modelling but a single-series CC deal is buildable without it.

Suggested phase additions (defer until after Phase 5):

- **Phase 6** — NLA + Required/Available Subordinated Amount + P-to-I reallocation (gaps 2, 3, 6 cluster naturally)
- **Phase 7** — Discount Option + Funding Account accumulation period semantics (gaps 1, 5)
- **Phase 8** — Multi-series master trusts with cross-series sharing (gap 4) — major effort
- **Phase 9** — Rolling-window triggers + pay-out events / early amortization deal state (gaps 7, 8)

These replace the "credit card master trusts" entry under "Asset classes still to cover" at the top of this doc.

Part 2 — IR reference: the schema and how to write a deal

This second half of the document covers the IR itself. Goal: make the IR both **highly reusable** (one schema for many asset classes) and **human-readable** (an analyst should be able to read the IR top-to-bottom and match it to the prospectus paragraph by paragraph).

IR overview

The IR is a JSON-serializable Pydantic schema that captures one deal's entire static structure in a single document. It is:

- **Source of truth.** The runtime executes the IR; the UI (Blockly canvas + Properties panel) edits the IR; saved deals persist as IR JSON; tests compare against the IR.
- **Versioned.** `schema_version: "1.0.0"` allows forward-compatible migrations.
- **Self-validating.** Pydantic enforces field types; the top-level `_validate_references` validator enforces cross-field consistency (every rule's `from_sources` and `to_targets` must reference declared bonds, accounts, fees, or built-in streams; every rule's `condition_trigger` must reference a declared trigger; every bond's `support_tranches` must reference real bonds; etc.).
- **Round-tripping.** IR → runtime → cashflow output is deterministic; IR → Blockly synthesizer → workspace → irGenerator → IR is meant to be lossless.

Top-level schema: `DealDefinition`

```
class DealDefinition(BaseModel):
    schema_version: str = "1.0.0"
    deal_name: str                        # "FNR 2006-018 (Group 1 + Group 2)"
    description: str = ""
    origination_date: date | None
    settlement_date: date | None

    # Structure
    bonds: list[BondDef]                  # All tranches, including pseudo bonds
    accounts: list[AccountDef]            # Reserves, prefunding, custodial
    fees: list[FeeDef]                    # Trustee, servicer, etc.
    triggers: list[TriggerNode]           # Conditional gates
    calculations: list[CalculationNode]   # Named expressions for triggers
    waterfall_rules: list[RuleNode]       # The actual priority of payments
    collateral_groups: list[CollateralGroupDef]  # Multi-pool deals

    # Solver / overrides / runtime extensions (see below)
    deal_knobs: dict[str, Any] = {}
```

`deal_knobs` — what's actually in there

`deal_knobs` is intentionally a free-form `dict[str, Any]` because it serves four distinct purposes. **Five reserved keys** have specific runtime meaning; everything else is a free scalar that gets injected into expression-evaluation context.

Key	Type	Purpose
`source_formulas`	`dict[str, str]`	Named per-period expressions that become first-class `from_source` tokens. Example: `{"NET_EXCESS": "ACT_INT - bond_coupon_total"}` makes `NET_EXCESS` referenceable in any rule's `from_sources`.
`balance_trackers`	`dict[str, str]`	Maps a residual / pseudo bond's name to a runtime quantity it should mirror. Example: `{"R": "collateral_balance"}` makes residual `R` carry a balance equal to outstanding pool balance each period. Used when a residual interest's economics track a non-bond quantity.
`orig_collat_bal_override`	`float`	Override for the original collateral balance used in trigger denominator calculations (when the natural pool balance differs from the trigger reference balance, e.g., tape excludes some loans).
`allow_negative_cashflow_math`	`bool`	Permit transient negative intermediate cash states during rule execution. Used for deals with negative-amortization streams (HECM, certain ARM cases) where cash conservation invariants need temporary relaxation.

Key	Type	Purpose
`<any_identifier>`	`int | float`	**Free expression-context globals.** Every numeric value with an identifier-safe key gets injected into the `expr` evaluation context for fee `amount_expr`, rule `max_amount_expr`, calculation `expression`, and trigger thresholds. Example: `{"servicing_fee_bps": 25.0}` lets a fee expression reference `servicing_fee_bps / 10000` directly.

The four functional roles, in plain language:

1. **Solver scratch-pad.** The solver writes proposed values into `deal_knobs.<name>` and re-runs the deal. `KnobSpec.knob_path` uses dot-paths like `deal_knobs.class_a_pctbal` to reach them.
2. **Expression-context globals.** Numeric values are auto-injected into the runtime expression evaluator so any expression-bearing field (`amount_expr`, `max_amount_expr`, `condition_expr`, trigger thresholds, calculation expressions) can reference them by bare name without declaring them as `CalculationNode`s.
3. **Runtime feature flags.** A few well-known boolean keys toggle runtime behavior (currently just `allow_negative_cashflow_math`).
4. **Runtime extension hooks.** `source_formulas` and `balance_trackers` are reserved-key escape hatches for capabilities that haven't been promoted to first-class IR fields yet. Both are candidates for elevation: `source_formulas` should become the proposed `ComputedAmountNode`; `balance_trackers` should become a `BondDef` field (`tracks_quantity: str | None`).

Migration intent for `deal_knobs`

Per the human-readability principle "no hidden runtime knobs," `deal_knobs` is a tension we accept temporarily. The direction of travel is to **shrink** the schemaless surface:

- `source_formulas` → graduate to `calculations: list[ComputedAmountNode]` (proposed addition B).
- `balance_trackers` → graduate to `BondDef.tracks_quantity` field.
- `orig_collat_bal_override` → graduate to a top-level `DealDefinition.original_collateral_balance: float | None`.
- `allow_negative_cashflow_math` → graduate to a `DealDefinition.runtime_options: RuntimeOptions` typed nested model (one field per flag).

After graduation, `deal_knobs` becomes purely #1 + #2: a typed scratch-pad of named scalar overrides for the solver and for expression evaluation, with everything behavioral promoted to real schema fields.

`BondDef` — every tranche the deal pays

A bond is anything that receives cash from the waterfall. This includes residual classes and "pseudo bonds" used as accounting sinks for fees.

```
class BondDef(BaseModel):
    name: str                            # "PA", "Class A-1", "M-1"
    # NOTE: tranche_type currently has 13 values mixing identity, schedule, coupon, and role.
    # Round 3 proposal G collapses this to TrancheKind (8 values:
    # CASH_PAY | PAC | TAC | IO | PO | Z | RESIDUAL | PSEUDO). PAC and TAC remain
    # because they're real economic identities; the validator enforces that
    # PAC/TAC kinds require a non-empty schedule_contract. SUPPORT becomes a
    # derived role; FLOATER/INVERSE_FLOATER live in coupon_type; SEQUENTIAL is
    # rule behavior, not bond identity; PAC_II is just a PAC with structural ordering.
    tranche_type: TrancheType            # SEQUENTIAL | PAC | PAC_II | TAC | SUPPORT | Z_BOND | ACCRETION_DIRECTED | FLOATER | INVERSE_FLOATER | IO | PO | PSEUDO | RESIDUAL
    # NOTE: tranche_behavior is fully redundant with the simplified TrancheKind + schedule + pay_mode.
    # Round 3 proposal G deletes this field entirely.
    tranche_behavior: TrancheBehavior    # SEQUENTIAL | PAC | TAC | Z
    is_bond: bool                        # False for residual/pseudo
    is_pseudo: bool                      # True for fee sinks

    # Coupon — Round 3 I: each rate field should accept a scalar OR a period-keyed
    # schedule (RateScheduleEntry list) so step-up / lockout coupons can be expressed.
    coupon_type: CouponType              # FIXED | FLOATING | INVERSE_FLOATING | ZERO
    coupon: float | None                 # Annual percent (5.5 = 5.5%)
    margin: float | None                 # Floater spread over index
    index_name: str | None               # SOFR | TERM_SOFR_1M | etc.
    cap: float | None                    # Floater rate cap
    floor: float | None                  # Floater rate floor
```

```
    # Sizing — Round 3 J: rename to `notional` and `notional_pct_of_collateral`.
    # "Notional" is the universal term (covers IOs which have notional but no principal flow)
    # and "size" is ambiguous.
    size_dollars: float | None             # → notional
    size_pct: float | None                 # → notional_pct_of_collateral (0..1, not 0..100)

    # Maturity / accrual
    maturity_date: date | None
    day_count: DayCount                    # 30/360 | ACT/360 | ACT/365 | ACT/ACT
    accrual_period: AccrualPeriod          # MONTHLY | QUARTERLY | SEMI_ANNUAL | ANNUAL

    # PAC / TAC parameters
    # Round 3 K: two-phase derivation — the speed fields are DESIGN-TIME inputs
    # (kept so the schedule can be re-derived if collateral / band assumptions
    # change). The runtime ONLY consumes `schedule_contract`. Derivation runs
    # at structuring-time on input change, caches the result here, and the
    # runtime reads the cache directly. TAC is the degenerate band where
    # low == high; `schedule_speed_target` is therefore redundant and is dropped.
    schedule_type: ScheduleType | None     # PAC | TAC | SUPPORT  (Round 3 G: drops SUPPORT — the kind tells
you)
    schedule_model_type: PrepayModelType  # PSA | CPR | ABS | CUSTOM_VECTOR  (design-time input)
    schedule_speed_low: float | None       # PAC lower PSA / TAC target  (design-time input)
    schedule_speed_high: float | None      # PAC upper PSA  (TAC: == low)  (design-time input)
    schedule_speed_target: float | None    # Round 3 K: dropped — redundant with low when TAC
    schedule_contract: list[dict]          # [{period, target_balance}] — runtime canonical, derived
    schedule_tolerance_bps: float | None

    # Tranche relationships — Round 3 H: collapse the next 6 fields into ONE
    # typed list `relations: list[TrancheRelation]` covering SUPPORTED_BY,
    # ACCRETES_TO, NOTIONAL_TRACKS, BALANCE_TRACKS, COUPON_INVERSE_OF,
    # COUPON_LEVERAGE_OF, MACR_EXCHANGE — full coverage of POs, IOs, inverse IOs,
    # inverse floaters, super floaters, MACR exchange classes.
    support_tranches: list[str]            # PAC support stack          → relations[?].SUPPORTED_BY
    supported_by_tranches: list[str]       # Z accretion targets         → relations[?].ACCRETES_TO
    parent_tranche: str | None             # IO/PO parent for tracking   → relations[?].NOTIONAL_TRACKS /
BALANCE_TRACKS
    relation_type: StructureRelation | None  # FLOATER_INVERSE | IO_PO | Z_ACCRUAL — Round 3 H expands to 7
values
    notional_ratio: float | None           # IO notional ratio           → relations[?].weights / leverage
    tracks_bonds: dict[str, list[str]] | None  # legacy IO/PO tracking  → relations[?].targets

    # Z-bond / accrual
    z_accrual_enabled: bool
    z_release_trigger: str | None
    accrual_start_period: int | None
    accrual_end_period: int | None

    # Multi-group
    group_id: str | None                   # "GROUP_1", "GROUP_2"

    # Solver knob flags
    solver_knob_coupon: bool
    solver_knob_size: bool

    pay_mode: PayMode                      # CASH_PAY | PIK
```

Reusability principle: one `BondDef` covers all bond types. Today the differentiation is split across `tranche_type` + `tranche_behavior` + schedule + accrual + tracking fields — that works but conflates intrinsic identity with rule-driven behavior. The Round 3 cleanup (proposals G + H) reduces this to one `TrancheKind` enum (the bond's identity), one `relations` list (its structural ties to other bonds), and the actual schedule / coupon / notional / accrual data. PAC-ness, TAC-ness, support status, and IO/PO tracking become consequences of those concrete fields — no separate behavior tag needed.

`RuleNode` — one waterfall step

```
class RuleNode(BaseModel):
    rule_id: str                           # "r_int_PA_pacI"
    rule_type: RuleType                    # PAY_INTEREST | PAY_PRINCIPAL | ...
    order: int                             # 0, 1, 2 ... priority
```

```
    from_sources: list[str]              # ["ACT_INT"] or ["GROUP_GROUP_1_ACT_PRIN"] or ["ReserveAcct"]
    to_targets: list[str]                # ["PA", "PB", "PC", "PD", "EO"] (bonds, accounts, named streams)
    payment_style: PaymentStyle          # SEQUENTIAL | PRO_RATA  (Round 3: CONCURRENT alias dropped)
    cap_mode: CapMode | None             # PLANNED | SCHEDULED | TARGETED | NONE

    max_amount_fixed: float | None       # Hard $ cap on this rule
    max_amount_expr: str | None          # Computed cap expression
    target_weights: list[float] | None   # SPLIT_CASH per-target weights (rule-level, not bond-level)

    condition_trigger: str | None        # Trigger name
    condition_invert: bool               # Run when trigger is FALSE
    condition_expr: str | None           # Custom condition expression

    reserve_account: str | None          # PAY_TO_RESERVE / PAY_FROM_RESERVE_*  (Round 3 M: redundant once
rules can source/target accounts directly)
    allow_negative_source: bool

    group_id: str | None                 # Multi-group routing
    description: str = ""
```

Reusability principle: every prospectus step maps to ONE `RuleNode`. The shape of the step (PAC schedule, sequential pay, pro-rata, fee, residual sweep, conditional pay) is conveyed by the *combination* of `rule_type` + `payment_style` + `cap_mode` + `condition_trigger`. **No new rule type for "PAY_PRINCIPAL_PAC_SCHEDULE"** or similar — those are just `PAY_PRINCIPAL` with `cap_mode=PLANNED` on a bond that has a `schedule_contract` (Round 3 G: the bond's PAC-ness is the schedule itself, not a separate `tranche_behavior` flag).

`RuleType` enum — what cash this rule moves

RuleType	Cash moved	Typical sources	Typical targets
`PAY_INTEREST`	Bond cash interest	typically `ACT_INT` or `CASH`; can be any account or stream	One or more bonds
`PAY_INTEREST_SHORTFALL`	Catch-up of unpaid interest	typically `ACT_INT`; can be any account	Bonds with unpaid coupon
`PAY_PRINCIPAL`	Principal	typically `ACT_PRIN` or `CASH`; can be any account or stream	One or more bonds
`PAY_WRITEDOWN`	Loss allocation	`LOSS`	Bonds (reverse seniority)
`PAY_FEE`	Fee	any account or stream	One fee payee
`PAY_TO_RESERVE`	Deposit to an account	any source	Any account
`PAY_FROM_RESERVE_INTEREST`	Withdraw from account, used for interest	An account	Bonds (interest shortfall)
`PAY_FROM_RESERVE_PRINCIPAL`	Withdraw from account, used for principal	An account	Bonds (principal acceleration)
`PAY_FROM_RESERVE`	Generic withdrawal from account	An account	Bonds or other targets
`PAY_RECOURSE_INTEREST`	Sponsor recourse covering interest	Recourse stream	Bonds (interest shortfall)
`PAY_RECOURSE_PRINCIPAL`	Sponsor recourse covering principal	Recourse stream	Bonds (principal acceleration)
`PAY_RESIDUAL`	Residual sweep	Any leftover stream	Residual classes
`SPLIT_CASH`	Stream plumbing	One source stream	N named virtual streams

(Round 3 cleanup, proposal M: the five reserve-specific rule types (`PAY_TO_RESERVE`, `PAY_FROM_RESERVE`, `PAY_FROM_RESERVE_INTEREST`, `PAY_FROM_RESERVE_PRINCIPAL`) and the recourse rule types (`PAY_RECOURSE_INTEREST`, `PAY_RECOURSE_PRINCIPAL`) conflate "what cash is moving" with "where it's coming from / going to." The proposed cleaner model: - `PAY_INTEREST` / `PAY_PRINCIPAL` accept any account or stream as their `from_sources`. So "pay interest from reserve" becomes `PAY_INTEREST from=[ReserveAcct]`. "Pay interest from sponsor recourse" becomes `PAY_INTEREST from=[RECOURSE_LINE]`. - `PAY_TO_RESERVE` is renamed to `PAY_TO_ACCOUNT` (accounts are not just reserves) and accepts any account as `to_targets`. - The "this rule covers an interest shortfall" semantic is moved from the rule type into a small `tracks_carryover_for: str` field on the rule, naming the bond whose shortfall ledger gets decremented.)

`PaymentStyle` — order semantics within a multi-target rule

Style	Behavior
`SEQUENTIAL`	Pay first target until its cap, then next, etc. ("In that order")

Style	Behavior
`PRO_RATA`	Pay all targets simultaneously by their balance / face / coupon weight

(Round 3 cleanup: `CONCURRENT` was a synonym of `PRO_RATA` and has been removed; older code referencing it should migrate to `PRO_RATA`.)

`CapMode` — schedule cap interpretation for PAY_PRINCIPAL

Mode	Stops paying when	Prospectus phrasing
`PLANNED`	Bond reaches its `schedule_contract` planned balance	"to its Planned Balance"
`SCHEDULED`	Bond reaches its scheduled (next-period) balance	"to its Scheduled Balance"
`TARGETED`	Bond reaches a target	"to its Targeted Balance"
`NONE`	Never (cleanup rule)	"without regard to its Planned Balance" / "until retired"

Built-in source/target tokens (reusable across asset classes)

Tokens that any rule's `from_sources` / `to_targets` can reference without explicit declaration. These are the canonical names enforced by `_validate_references` in `schemas/ir.py`.

Current vocabulary (post Phase 1c, May 2026)

Token	Meaning	BMA field
`CASH`	Combined gross collateral cashflow	`act_cash` (= `act_prin + act_int`)
`ACT_INT`	Pool interest stream only	`act_int`
`ACT_PRIN`	Pool principal stream only	`act_prin` (= `act_am + vol_prepay`)
`LOSS`	Pool loss stream (for writedown rules)	`prin_loss`
`GROUP_<id>_CASH`	Combined cashflow for collateral group `<id>`	per-group `act_cash`
`GROUP_<id>_ACT_INT`	Interest stream for collateral group `<id>`	per-group `act_int`
`GROUP_<id>_ACT_PRIN`	Principal stream for collateral group `<id>`	per-group `act_prin`
`GROUP_<id>_LOSS`	Loss stream for collateral group `<id>`	per-group `prin_loss`

When a rule declares `group_id`, the bare tokens (`CASH`, `ACT_INT`, `ACT_PRIN`, `LOSS`) are auto-prefixed with `GROUP_<id>_` at compile time. So a multi-group rule can write `from_sources: ["ACT_INT"]` and have it resolve to the right group automatically.

(Round 3 cleanup notes: - Phase 1c (May 2026) renamed `INT_CASH` → `ACT_INT` and `PRIN_CASH` → `ACT_PRIN` to align IR vocabulary with the BMA-native cashflow field names (`BMAActualCashflow.act_int`, `act_prin`). The `COLLATERAL` and `GROUP_<id>_COLLATERAL` aliases for `CASH` were dropped; they were redundant. - The validator does NOT accept old names — the rename is hard. All in-tree fixtures, tests, and frontend code were migrated in the same commit.)

Planned forward direction — `COLL_*` taxonomy (see proposal below)

The CC research (Round 3) and the user-driven taxonomy review (May 2026) converged on a more granular `COLL_*` naming scheme that exposes finer-grained BMA cashflow primitives:

Token	Meaning	BMA field
Combined streams (most common)		
`COLL_CASH`	Combined gross collateral cashflow	`act_cash`
`COLL_PRIN`	All principal cash	`act_prin`
`COLL_INT`	Gross interest collected (before servicing)	`act_int`
`COLL_LOSS`	Realized principal loss (default)	`prin_loss`
Principal decomposition		
`COLL_PRIN_SCH`	Contractual amortization	`act_am`
`COLL_PRIN_PPAY`	Voluntary prepayments	`vol_prepay`
`COLL_PRIN_REC`	Liquidation recoveries (RMBS)	`prin_recov`
`COLL_PRIN_ADV`	Servicer principal advances (in)	`adv_prin`
`COLL_PRIN_ADV_REIMB`	Reimbursement (out)	`adv_reimbursed_prin`
Interest decomposition		

Token	Meaning	BMA field
`COLL_INT_ADV`	Servicer interest advances (in)	`adv_int`
`COLL_INT_ADV_REIMB`	Reimbursement (out)	`adv_reimbursed_int`
Loss decomposition		
`COLL_LOSS_PRIN`	Realized principal loss	`prin_loss`
`COLL_LOSS_INT`	Interest lost to defaults	`lost_int`
`COLL_LOSS_ADV`	Unreimbursable advances	`adv_unrecoverable`
Fee streams (rare)		
`COLL_SVC`	Servicing fee accrued	`svc_billed`
Balance references (read-only, not waterfall sources)		
`COLL_PERF_BAL`	Performing balance	`perf_bal`
`COLL_FCL_BAL`	Foreclosure balance	`fcl`
`COLL_TOTAL_BAL`	Total outstanding	`total_bal` (= `perf_bal + fcl`)

Key naming decisions:

1. **`COLL_*` everywhere.** Single prefix; per-group form reads `GROUP_<id>_COLL_PRIN`. Asset-class neutral (RMBS, CC, auto, CRT all use the same prefix).
2. **Bare form means total.** No `_TOT` suffix. `COLL_PRIN` is the total; `COLL_PRIN_SCH` / `COLL_PRIN_PPAY` are decompositions.
3. **`COLL_LOSS` defaults to principal loss.** Most waterfalls only trigger writedowns from realized principal losses; explicit `COLL_LOSS_INT` / `COLL_LOSS_ADV` for the rarer cases.
4. **`COLL_INT` is gross of servicing.** Servicing is paid via a `PAY_FEE` rule from `COLL_INT`, mirroring prospectus language ("the trust receives X% interest and pays the master servicer Y bps"). This matches both RMBS and CC waterfall conventions.
5. **Balance tokens are read-only.** Reference them in `max_amount_expr`, trigger metrics, fee bases — but the validator should reject them as `from_sources` / `to_targets` on cash-moving rules.
6. **Asset-class neutrality of decomposition tokens.** Recoveries in RMBS are principal (`COLL_PRIN_REC`); recoveries in CC fold into finance charges (`COLL_INT`). The token vocabulary doesn't change — only which tokens are populated by the loan-level engine.

This rename is the proposed Phase 2 of the IR cleanup (post-Phase 1f migration). Until then, the current `ACT_*` vocabulary stays canonical.

Other elements

`AccountDef` — named cash buckets (reserves, prefunding, etc.)

```
class AccountDef(BaseModel):
    name: str                             # "Reserve_Account"
    # Current code: RESERVE | PREFUNDING | REVOLVING | PAYMENT | SPREAD_ACCOUNT
    # VERIFIED against runtime.py: account_type is a passthrough display label only.
    # The runtime stores it at init (line 324) and copies it to output (line 1611);
    # there is NO `if account_type == X` branch anywhere. Round 3 O renames to
    # AccountCategory. Behavior comes from minimum_basis + the rules that touch
    # the account; minimum_basis is now (post-Round 3 Q fix) honored at runtime
    # for both starting and required-minimum calculations.
    account_type: AccountType             # RESERVE | PREFUNDING | REVOLVING | PAYMENT | SPREAD_ACCOUNT
    starting_amount: float                # $ amount at closing
    starting_pct: float | None            # OR % of <basis quantity> at period 0
    starting_basis: MinimumBasis          # FIXED_DOLLAR | COLLATERAL_BALANCE | NOTE_BALANCE |
ORIGINAL_COLLATERAL
                                          # — honored at runtime as of May 2026 (Round 3 Q fix)
    minimum_amount: float                 # Dollar floor; combined with the pct-derived floor via max()
    minimum_pct: float | None             # OR % of <basis quantity> per minimum_basis
    minimum_basis: MinimumBasis           # FIXED_DOLLAR (constant $) | COLLATERAL_BALANCE (steps down with
pool) |
                                          # NOTE_BALANCE (steps down with notes) | ORIGINAL_COLLATERAL
(constant pct of orig)
                                          # — honored at runtime as of May 2026 (Round 3 Q fix)
```

`FeeDef` — periodic fee paid to a payee

```
class FeeDef(BaseModel):
    name: str                             # "TRUSTEE", "SERVICER"
    basis_type: FeeBasisType              # FIXED_DOLLAR | COLLATERAL_BALANCE | PER_LOAN
    amount: float                         # $ amount per period (FIXED_DOLLAR)
    amount_expr: str | None               # OR computed expression
    rate: float | None                    # Annual rate (COLLATERAL_BALANCE)
    rate_expr: str | None
    minimum: float                        # Floor on the fee
    frequency: FeeFrequency               # MONTHLY | QUARTERLY | ANNUAL
    cumulative: bool                      # Track unpaid fees as carryover
```

`TriggerNode` — conditional gate

```
class TriggerNode(BaseModel):
    name: str                             # "CumLossTrigger"
    metric_type: TriggerMetricType        # CUMULATIVE_LOSS | CUMULATIVE_DEFAULT | DELINQUENCY_RATE | OC_TEST
| IC_TEST | CUSTOM
    threshold_value: float | None
    threshold_schedule: list[float] | None # Per-period threshold (e.g., RMBS time-stepped triggers)
    calculation_ref: str | None           # Pointer to a CalculationNode for dynamic thresholds
    comparison_ref: str | None            # ">" / ">=" / etc.
    cure_periods: int | None              # How many clean periods to clear the trigger
```

`CalculationNode` — named expressions

```
class CalculationNode(BaseModel):
    name: str                             # "cum_loss_pct"
    expression: str                       # "sum(loss[0:i+1]) / orig_collat_bal"
    description: str
```

Used by triggers (and in proposed `ComputedAmountNode`) to compute per-period scalars. Supports a safe subset of Python: arithmetic, min/max/abs, references to bond/account/pool state.

`CollateralGroupDef` — multi-pool deals

```
class CollateralGroupDef(BaseModel):
    group_id: str                         # "GROUP_1"
    label: str                            # Human-readable
    description: str
```

Activates per-group cash routing in the runtime. When non-empty, every non-pseudo bond and every cashflow-touching rule must declare `group_id`.

Worked example: FNR 2006-018 Group 1, written in IR

The prospectus says (verbatim, paraphrased for brevity):

> *Group 1 cash flow priority of payments: 1. Pay interest on all cash-paying bonds. 2. Pay principal of Aggregate Group I to its Planned Balance (PA → PB → PC → PD → EO sequential). 3. Pay principal of Aggregate Group II to its Planned Balance (TA → TB sequential). 4. Pay principal to Z to zero. 5. Distribute 95.65% of remaining principal to support sequential (WA → WB → WC → WD → WE → WG); 4.35% to PO. 6. Pay principal of Aggregate Group II to zero (without regard to planned balance). 7. Pay principal of Aggregate Group I to zero (without regard to planned balance). 8. Sweep remaining cash to residual.*

Below is the **same waterfall expressed compactly in IR** (one rule per prospectus step). Every rule has a stable `rule_id` that matches the prospectus phrasing so an analyst can search the IR and find the corresponding step:

```
deal_name: "FNR 2006-018 Group 1"
collateral_groups:
  - group_id: GROUP_1
    label: "Group 1 (PAC + Z + Support)"

bonds:
  # Aggregate Group I (PAC I + IO/PO)
  - { name: PA,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1,
      coupon: 5.5, size_dollars: 33710000, schedule_contract: [...],
      support_tranches: [WA, WB, WC, WD, WE, WG, PO] }
```

```yaml
  - { name: PB,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1, ... }
  - { name: PC,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1, ... }
  - { name: PD,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1, ... }
  - { name: EO,  tranche_type: PO,     coupon_type: ZERO,     group_id: GROUP_1, ... }
  - { name: EI,  tranche_type: IO,     tracks_bonds: { balance: [PA, PB, PC, PD] }, ... }

  # Aggregate Group II (PAC II / accretion-directed)
  - { name: TA,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1, ... }
  - { name: TB,  tranche_type: PAC,    tranche_behavior: PAC, group_id: GROUP_1, ... }

  # Z-bond (Aggregate Group II support)
  - { name: Z,   tranche_type: Z_BOND, tranche_behavior: Z, pay_mode: PIK,
      group_id: GROUP_1, z_accrual_enabled: true, supported_by_tranches: [TA, TB] }

  # Support tranches
  - { name: WA,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WB,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WC,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WD,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WE,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WG,  tranche_type: SUPPORT, group_id: GROUP_1, ... }

  # Support PO (4.35% of support cash)
  - { name: PO,  tranche_type: PO,     coupon_type: ZERO, group_id: GROUP_1, ... }

  # Residual
  - { name: R,   tranche_type: RESIDUAL, is_pseudo: true }

waterfall_rules:
  # Step 1 — Interest cascade (one rule, all cash-paying bonds in priority order)
  - rule_id: r_int_cascade
    rule_type: PAY_INTEREST
    order: 0
    group_id: GROUP_1
    from_sources: [ACT_INT]
    to_targets: [PA, PB, PC, PD, TA, TB, EI, WA, WB, WC, WD, WE, WG]
    payment_style: SEQUENTIAL
    description: "Pay accrued bond coupon. Z is PIK; its coupon is capitalized."

  # Step 2 — PAC I to its Planned Balance (one rule, all PAC I bonds in priority)
  - rule_id: r_prin_pac_i_planned
    rule_type: PAY_PRINCIPAL
    order: 1
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [PA, PB, PC, PD, EO]
    payment_style: SEQUENTIAL
    cap_mode: PLANNED
    description: "Aggregate Group I to its Planned Balance"

  # Step 3 — PAC II to its Planned Balance
  - rule_id: r_prin_pac_ii_planned
    rule_type: PAY_PRINCIPAL
    order: 2
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [TA, TB]
    payment_style: SEQUENTIAL
    cap_mode: PLANNED
    description: "Aggregate Group II to its Planned Balance"

  # Step 4 — Z-bond
  - rule_id: r_prin_Z
    rule_type: PAY_PRINCIPAL
    order: 3
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [Z]
```

```yaml
    payment_style: SEQUENTIAL
    description: "Z to zero"

  # Step 5a — 95.65 / 4.35 face-weighted split
  - rule_id: r_supp_split
    rule_type: SPLIT_CASH
    order: 4
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [WAWG_BUCKET, PO_BUCKET]
    target_weights: [0.956521694276, 0.043478305724]
    description: "Face-weighted split of remaining principal: 95.65% to WA-WG, 4.35% to PO"

  # Step 5b — WA-WG cascade (multi-target sequential)
  - rule_id: r_pay_wawg
    rule_type: PAY_PRINCIPAL
    order: 5
    group_id: GROUP_1
    from_sources: [WAWG_BUCKET]
    to_targets: [WA, WB, WC, WD, WE, WG]
    payment_style: SEQUENTIAL

  # Step 5c — Support PO
  - rule_id: r_prin_PO
    rule_type: PAY_PRINCIPAL
    order: 6
    group_id: GROUP_1
    from_sources: [PO_BUCKET]
    to_targets: [PO]

  # Step 5d — Sweep leftover bucket cash back into ACT_PRIN
  - rule_id: r_supp_sweep_back
    rule_type: SPLIT_CASH
    order: 7
    group_id: GROUP_1
    from_sources: [WAWG_BUCKET, PO_BUCKET]
    to_targets: [ACT_PRIN]
    target_weights: [1.0]

  # Step 6 — PAC II cleanup
  - rule_id: r_prin_pac_ii_uncapped
    rule_type: PAY_PRINCIPAL
    order: 8
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [TA, TB]
    payment_style: SEQUENTIAL
    cap_mode: NONE
    description: "Aggregate Group II to zero, without regard to Planned Balance"

  # Step 7 — PAC I cleanup
  - rule_id: r_prin_pac_i_uncapped
    rule_type: PAY_PRINCIPAL
    order: 9
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [PA, PB, PC, PD, EO]
    payment_style: SEQUENTIAL
    cap_mode: NONE

  # Step 7b — Support cleanup (in case face-weighted split left tail residue)
  - rule_id: r_prin_supports_uncapped
    rule_type: PAY_PRINCIPAL
    order: 10
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [WA, WB, WC, WD, WE, WG, PO]
    payment_style: SEQUENTIAL
```

```yaml
    cap_mode: NONE

  # Step 8 — Residual
  - rule_id: r_resid_int
    rule_type: PAY_RESIDUAL
    order: 11
    group_id: GROUP_1
    from_sources: [ACT_INT]
    to_targets: [R]
    payment_style: SEQUENTIAL

  - rule_id: r_resid_prin
    rule_type: PAY_RESIDUAL
    order: 12
    group_id: GROUP_1
    from_sources: [ACT_PRIN]
    to_targets: [R]
    payment_style: SEQUENTIAL
```

Rule count: 13 (one per logical waterfall step), versus the current FNR fixture's **35 rules** (one per bond per step). The behavior is identical — the runtime walks `to_targets` sequentially with the right cap_mode — but the IR is now readable as prospectus paragraphs.

Reusability principles

1. **One bond schema covers all bond types.** The `BondDef` shape accommodates PAC, TAC, Z, IO, PO, sequential, support, residual, pseudo via the `tranche_type` + `tranche_behavior` + schedule / accrual / tracking fields. **Don't add a `PACBondDef` or `ZBondDef` subclass** — encode the variation in the existing fields.
2. **One rule schema covers all rule types.** Most prospectus structural variation is captured by the *combination* of `rule_type` + `payment_style` + `cap_mode` + `condition_trigger`. PAC / TAC / cleanup / mezz cascades / interest waterfalls / principal waterfalls / fee priorities all fit. **Don't add `PAY_PRINCIPAL_PAC_SCHEDULE` or similar specialized rule types** — encode the variation in `cap_mode` and the bond's `tranche_behavior`.
3. **One rule per prospectus step, not one per bond.** The prospectus says "Pay sequentially to PA, PB, PC, PD, EO until their planned balances." That is **ONE rule** with `to_targets: [PA, PB, PC, PD, EO]` and `cap_mode: PLANNED`. Do NOT fragment it into 5 rules.
4. **Built-in tokens are reused everywhere.** `CASH` / `ACT_INT` / `ACT_PRIN` / `LOSS` are universal. Multi-group deals add `GROUP_<id>_*` prefixes which the runtime auto-resolves when `group_id` is set on the rule.
5. **Asset-class differences are parametric, not structural.** Prime auto vs subprime auto: same grammar, more classes / more named amounts / lower fee cap. Agency MBS vs non-agency RMBS: different feature mix (PAC vs OC/stepdown), but the same primitives.

Human-readability principles

1. **Stable, descriptive `rule_id`s that match the prospectus.** `r_prin_pac_i_planned` reads as "rule, principal, PAC I, to planned balance" — a person can scan the IR and match it to the prospectus paragraph. Avoid synthetic ids like `r_principal_0001` that lose intent.
2. **`description` field on every rule.** One sentence, ideally verbatim from the prospectus. The IR should be readable on its own without referring back to the source document.
3. **Group IR top-level lists by purpose, not alphabetically.** `bonds` ordered by seniority; `waterfall_rules` ordered by priority (matching `order`); `triggers` ordered by deal-state relevance. The IR document reads top-to-bottom as the prospectus reads.
4. **Comments allowed via `description` even where Pydantic doesn't normally permit.** Every node has a `description: str` field. Use it. Especially for:
   - Rules: paste the prospectus phrasing verbatim.
   - Bonds: note the rationale for any non-obvious field (e.g., why `support_tranches` is set as it is).
   - Triggers: explain the threshold table and cure logic.
5. **Compact over verbose where math is identical.** If a rule could be written as one multi-target rule or as N single-target rules, prefer the compact form because it matches the prospectus. The runtime produces identical output either way; the compact form is far easier to read and edit.
6. **Avoid hidden runtime knobs.** If a deal needs an exotic behavior (Z accrual, schedule cap mode, face-weighted split), make it visible in the IR via existing fields (`cap_mode`, `target_weights`, `pay_mode=PIK`). Do not bury it in `deal_knobs` or in code branches in the runtime.

7. **One DealDefinition file per real-world deal.** A single, self-contained, version-controlled JSON / YAML / Python factory. The fixture for FNR 2006-018 is one file: `tests/fixtures/fnr_2006_018/deal_definition.py`. The full combined Group 1 + Group 2 deal is one `DealDefinition` with multiple `collateral_groups`, not two separate files.

Anti-patterns to avoid

Anti-pattern	Better approach
One rule per bond ("`r_int_PA`, `r_int_PB`, …")	One rule per waterfall step (`r_int_cascade` with all bonds)
New `RuleType` for each prospectus phrase	Existing rule types + `cap_mode` + `payment_style`
Subclassing `BondDef` for PAC / Z / IO	Set `tranche_type` + `tranche_behavior` + `schedule_contract` / `tracks_bonds`
Ad-hoc behavioral key in `deal_knobs` (not one of the 5 reserved keys)	Visible IR fields (`target_weights`, `cap_mode`, `pay_mode`) — promote new behavior to a real schema field
Synthetic `rule_id` like `r_001`	Descriptive `r_prin_pac_i_planned`
Empty `description` field	Verbatim prospectus phrasing
Two `DealDefinition` files for one prospectus deal with multiple groups	One DealDefinition with multiple `collateral_groups`
Trigger logic inline in expressions	Named `TriggerNode` referenced by `condition_trigger`
Computed amount inline in `max_amount_expr`	Named `CalculationNode` (or proposed `ComputedAmountNode`)